

```
import numpy as np
import pandas as pd
import time
import tkinter as tk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
import threading
from scipy.signal import savgol_filter
import scipy as scp
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from binance import Client
from google.cloud import bigquery
from statsmodels.tsa.arima.model import ARIMA
import yfinance as yf
import requests
import smtplib
from email.mime.text import MIMEText
import logging
import os

# Configuration

Lo=1
aDecm=3
Ngroup=3
Nproc=Ngroup*(os.cpu_count()-1)

NIter=1

#####
##https://grok.com/share/c2hhcm0tMg%3D%3D_973e354b-3dd8-4a23-9011-32258d0eb7ce
# Environment variables for Binance, BigQuery, and notifications
API_KEY = os.getenv("BINANCE_API_KEY", "YOUR_BINANCE_KEY")
API_SECRET = os.getenv("BINANCE_API_SECRET", "YOUR_SECRET")
SYMBOL = os.getenv("SYMBOL", "BTC-USD")
TRAILING_STOP_PCT = 0.02
STOP_LOSS_PCT = 0.02
TAKE_PROFIT_PCT = 0.05
START_HOUR = 8
END_HOUR = 20
WINDOW_SIZE = 60
DETERM = 0.1
EMAIL_SENDER = os.getenv("EMAIL_SENDER", "you@example.com")
EMAIL_PASSWORD = os.getenv("EMAIL_PASSWORD", "password")
EMAIL_RECEIVER = os.getenv("EMAIL_RECEIVER", "receiver@example.com")
SMTP_SERVER = os.getenv("SMTP_SERVER", "smtp.example.com")
SMTP_PORT = 587
TELEGRAM_TOKEN = os.getenv("TELEGRAM_TOKEN", "bot_token")
TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "chat_id")
BQ_PROJECT = os.getenv("GOOGLE_CLOUD_PROJECT")
BQ_DATASET = os.getenv("BQ_DATASET", "ralfidataset")
BQ_TABLE = os.getenv("BQ_TABLE", "btc_prices")

# Logging setup
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

# Ensure GOOGLE_APPLICATION_CREDENTIALS is set for BigQuery authentication
if not os.getenv("GOOGLE_APPLICATION_CREDENTIALS"):
    logging.warning("GOOGLE_APPLICATION_CREDENTIALS not set. Please set it to the path of your service account key JSON.")
    raise ValueError("BigQuery authentication requires GOOGLE_APPLICATION_CREDENTIALS environment variable.")

#####
import time as tm
import hickle as hkl
import multiprocessing as mp
import warnings
from operator import itemgetter
from random import sample
from random import Random
import struct

wrkdir = ""
GetSCV=0
Numproc=mp.cpu_count()-1
MxTime=0.5*60*60 # 2 hours

class BaseRandom( Random ):

    def __init__(self, _seed=None):

        super().__init__( _seed ) ## this call creates attribute self._value and sets it

    def seed(self, _seed=None):

        try:
            self.setstate( _seed )
        except:
            super().seed( _seed )

    def __call__(self, _max=1.0):

        return self.uniform( 0.0, _max )

class BaseLCG( BaseRandom ):

    def __init__(self, _seedState=None):

        super().__init__( _seedState ) # this call creates attribute self._value and sets it

    def random(self):

        raise NotImplementedError

    def getstate(self):

        return self._value

    def setstate(self, _state):

        raise NotImplementedError

class FastRand63( BaseLCG ):
#Copyright (c) 2016-2019 Philippe Schmouker, schmouk (at) typee.ovh
    def random(self):

        self._value = (9219741426499971445 * self._value + 1) & 0x7fffffffffffffff
        return self._value / 9223372036854775808.0

    def setstate(self, _state):

        if isinstance( _state, int ):
            self._value = _state & 0x7fffffffffffffff

        elif isinstance( _state, float ):
            # transforms passed initial seed from float to integer
            if _state < 0.0 :
                _state = -_state

            if _state >= 1.0:
                self._value = int(_state+0.5) & 0x7fffffffffffffff
            else:
                self._value = int(_state*0x8000000000000000) & 0x7fffffffffffffff
```



```

        else:
            t = int(time.time() * 1000.0)
            self._value = t & 0xffffffff
            self._value += (t & 0xffff000000) << 8
            self._value += (t & 0x00ff0000) << 24
            self._value += (t & 0x0000ff00) << 40
            self._value += (t & 0x000000fe) << 63

class BaseLFib64( BaseRandom ):

    def __init__(self, _seedState=None):
        super().__init__( _seedState )

    def random(self):
        raise NotImplementedError

    def getstate(self):
        return (self._list[:], self._index)

    def setstate(self, _seedState):
        if not isinstance( _seedState, tuple ):
            self._initIndex( 0 )
            self._initList( _seedState )

        elif len( _seedState ) < 2:
            self._initIndex( 0 )
            self._initList( _seedState[0] )

        else:
            self._initIndex( _seedState[1] )
            self._list = _seedState[0][:]

    def _initIndex(self, _index):
        try:
            self._index = int( _index ) % self._LIST_SIZE
        except:
            self._index = 0

    def _initList(self, _initialSeed=None):
        myRand = FastRand63( _initialSeed )

        def _getValue( _dummy ):
            myRand()
            v = myRand._value <= 1
            return v + 1 if myRand() >= 0.5 else 0

        self._list = list( map( _getValue, range(self._LIST_SIZE) ) )

class LFib1340( BaseLFib64 ):
    #Copyright (c) 2016-2019 Philippe Schmouker, schmouk (at) typee.ovh

    _LIST_SIZE = 1279 # this 'LFib(2^64, 1279, 861, +)' generator is based on a suite containing 1279 integers

    def random(self):
        """
        This is the core of the pseudo-random generator.
        Returned values are within [0.0, 1.0).
        """
        # evaluates indexes in suite for the i-861 and i-1279 -th values
        k861 = self._index-861
        if k861 < 0:
            k861 += LFib1340._LIST_SIZE

        # then evaluates current value
        myValue = (self._list[k861] + self._list[self._index]) & 0xffffffffffffffff
        self._list[self._index] = myValue

        # next index
        self._index = (self._index+1) % LFib1340._LIST_SIZE

        # then returns float value within [0.0, 1.0)
        return myValue / 18446744073709551616.0

def XFilterB(dQ2,key=1,key2=1,key3=1):
    #Copyright (c) 2011 Vadim Chernov, ptnt. 2011612714RF
    if key>0:
        Np=len(dQ2)
        Nf=len(dQ2[0])
        SdQj=np.ones(Np,float)
        znakSdQj=np.ones(Np,float)

    if key2>0:
        srdQ2=np.mean(dQ2,axis=0)
        dQ2=dQ2-srdQ2
        for i in range(Np):
            znakSdQj[i]=np.mean(dQ2[i])
            znakSdQj=((znakSdQj<0)-0.5)*2*(-1)

    SSdQ=np.std(dQ2)

    for i in range(Np):
        if key3>0:
            SdQj[i]=np.std(dQ2[i])
            dQ2[i] = dQ2[i] * znakSdQj[i]
            if SdQj[i]==0:
                dQ2[i]=np.zeros(Nf,float)
            else:
                dQ2[i]=dQ2[i] / SdQj[i]

    SdQ=np.zeros(Nf,float)
    for j in range(Nf):
        SdQ[j]=np.mean(dQ2[:,j])
    sSdQ=np.std(SdQ)
    for i in range(Np):
        SdQj_ = np.std(dQ2[i] - SdQ)
        SdQj__ = np.std(dQ2[i])
        if SdQj_ >0 and sSdQ>0:
            dQ2[i] = dQ2[i] +SdQ * (SdQj_ / sSdQ - 1)
            # dQ2[i] = dQ2[i] * (1+sSdQ / SdQj__) / 2
            dQ2[i] = dQ2[i] *znakSdQj[i] #* SdQj[i]
        else:
            dQ2[i]=np.zeros(Nf,float)

    for i in range(Np):
        dQ2[i] = dQ2[i]* SdQj[i]

    SSdQ =np.std(dQ2)
    dQ2=dQ2*SSdQ/SSdQ_

    if key2>0:
        for i in range(Np):
            dQ2[i]=dQ2[i]+srdQ2

    return dQ2

MaxTemp=90

```



```

t_cache = {} # Global cache for T-Matrix and Psi

import numpy as np
import logging
from scipy.signal import welch
from scipy.stats import wasserstein_distance
from sklearn.covariance import LedoitWolf
import pywt

# === CONFIGURATION ===
USE_KOLMOGOROV_ENHANCEMENTS = True
FAST_MODE = True
KOLMOGOROV_WEIGHTS = {
    'kc': 0.15, 'spectral': 0.05, 'wavelet': 0.10, 'mahal': 0.15,
    'kan': 0.15, 'graph': 0.10, 'wass': 0.05, 'ae': 0.10, 'hurst': 0.05
}

# === VECTORIZED & FAST VERSIONS ===
def fast_lz_complexity(dd, bins=5):
    n, m = dd.shape
    if m < 4: return np.zeros(n)
    mins = dd.min(axis=1, keepdims=True)
    maxs = dd.max(axis=1, keepdims=True) + 1e-12
    binned = ((dd - mins) / (maxs - mins) * (bins-1)).astype(int)
    complexity = np.zeros(n)
    for i in range(n):
        row = binned[i]
        if m < 3:
            complexity[i] = 1.0
            continue
        grams = np.lib.stride_tricks.sliding_window_view(row, 3)
        complexity[i] = len(np.unique(grams, axis=0))
    return np.clip(complexity / (m//3 + 1), 0, 1)

def fast_spectral_entropy(dd, fs=1.0):
    n, m = dd.shape
    if m < 8: return np.zeros(n)
    entropy = np.zeros(n)
    segment = min(64, m//4)
    for i in range(n):
        f, Pxx = welch(dd[i], fs=fs, nperseg=segment, noverlap=0)
        Pxx = Pxx[Pxx > 1e-12]
        Pxx /= Pxx.sum()
        entropy[i] = -np.sum(Pxx * np.log2(Pxx))
    return np.clip(entropy / np.log2(segment//2 + 1), 0, 1)

def fast_wavelet_energy(dd, level=2):
    n, m = dd.shape
    if m < 8: return np.zeros(n)
    energy = np.zeros(n)
    for i in range(n):
        try:
            wp = pywt.WaveletPacket(dd[i], 'db1', mode='symmetric', maxlevel=level)
            nodes = wp.get_level(level)
            energies = [np.sum(n.data**2) for n in nodes]
            total = sum(energies) + 1e-12
            energy[i] = energies[-1] / total
        except:
            energy[i] = 0.0
    return energy

def fast_mahal_robust(dd):
    n, m = dd.shape
    if n < 3 or m < 2: return np.zeros(n)
    try:
        if n > 50:
            idx = np.random.choice(n, 50, replace=False)
            sample = dd[idx]
        else:
            sample = dd
            lw = LedoitWolf().fit(sample)
            cov = lw.covariance_ + 1e-6 * np.eye(m)
            mean = lw.location
            dist = np.zeros(n)
            for i in range(n):
                delta = dd[i] - mean
                dist[i] = np.sqrt(delta @ np.linalg.pinv(cov) @ delta)
            return np.clip(dist / (np.percentile(dist, 95) + 1e-12), 0, 1)
    except:
        return np.zeros(n)

def fast_kan_layer(dd):
    variances = np.var(dd, axis=1)
    return 0.1 * (variances**2 + 0.1 * np.sin(variances))

def fast_graph_degree(dd, tau=0.8):
    n = dd.shape[0]
    if n < 2: return np.zeros(n)
    norms = np.linalg.norm(dd, axis=1, keepdims=True)
    C = np.abs(dd @ dd.T) / (norms @ norms.T + 1e-12)
    np.fill_diagonal(C, 0)
    G = (C > tau).astype(int)
    deg = G.sum(axis=1)
    return 1 - deg / (deg.max() + 1e-12)

def fast_wasserstein(dd):
    n, m = dd.shape
    if n == 0: return np.zeros(0)
    template = np.mean(dd, axis=0)
    dists = np.array([wasserstein_distance(dd[i], template) for i in range(min(n, 20))])
    if n <= 20:
        return dists / (dists.max() + 1e-12)
    return np.clip(dists.mean() * np.ones(n), 0, 1)

def fast_ae_error(dd, latent=4):
    n, m = dd.shape
    if n < 2 or m < 2: return np.zeros(n)
    try:
        U, S, Vt = np.linalg.svd(dd, full_matrices=False)
        proj = U[:, :latent] @ np.diag(S[:latent]) @ Vt[:, :latent, :]
        err = np.sum((dd - proj)**2, axis=1)
        return np.sqrt(err) / (np.sqrt(err).max() + 1e-12)
    except:
        return np.zeros(n)

def fast_hurst(dd):
    n, m = dd.shape
    if m < 20:
        if return np.full(n, 0.5)
    h = np.zeros(n)
    max_lag = min(20, m // 4)
    lags = np.arange(2, max_lag + 1)
    log_lags = np.log(lags)
    for i in range(n):
        ts = dd[i]
        tau = np.array([np.std(ts[lag:] - ts[:-lag]) for lag in lags])
        if len(tau) < 2 or np.any(tau <= 0):
            h[i] = 0.5
            continue
        log_tau = np.log(tau)
        valid = np.isfinite(log_tau)
        if valid.sum() < 2:

```



```

        h[i] = 0.5
        continue
    poly = np.polyfit(log_lags[valid], log_tau[valid], 1)
    h[i] = np.clip(poly[0], 0.1, 2.0)
    return 1 - np.abs(h - 0.5) / 0.5

def fast_kolmogorov_ensemble(dd_pre):
    if not FAST_MODE or not USE_KOLMOGOROV_ENHANCEMENTS:
        return np.zeros(dd_pre.shape[0])

    scores = {}
    w = KOLMOGOROV_WEIGHTS
    if w['kc'] > 0: scores['kc'] = fast_lz_complexity(dd_pre)
    if w['spectral'] > 0: scores['spectral'] = fast_spectral_entropy(dd_pre)
    if w['wavelet'] > 0: scores['wavelet'] = fast_wavelet_energy(dd_pre)
    if w['mahal'] > 0: scores['mahal'] = fast_mahal_robust(dd_pre)
    if w['kan'] > 0: scores['kan'] = fast_kan_layer(dd_pre)
    if w['graph'] > 0: scores['graph'] = fast_graph_degree(dd_pre)
    if w['wass'] > 0: scores['wass'] = fast_wasserstein(dd_pre)
    if w['ae'] > 0: scores['ae'] = fast_ae_error(dd_pre)
    if w['hurst'] > 0: scores['hurst'] = fast_hurst(dd_pre)

    if not scores: return np.zeros(dd_pre.shape[0])

    weighted = np.zeros(dd_pre.shape[0])
    total_w = sum(w.get(k, 0) for k in scores)
    for name, arr in scores.items():
        weighted += w.get(name, 0) * arr
    return weighted / (total_w + 1e-12)

# === CORE FILTERING FUNCTIONS ===
def hybrid_preprocess(dd, intensity=0.1, alpha=0.5, p_combined=None, original_shape=None):
    if not isinstance(dd, np.ndarray) or dd.ndim != 2:
        raise ValueError("Input must be a 2D NumPy array")
    n, m = dd.shape
    if n == 0 or m == 0:
        return np.ones(n), np.zeros_like(dd)

    max_abs_dd = np.max(np.abs(dd))
    dd_norm = dd / (max_abs_dd + 1e-10) if max_abs_dd > 0 else np.zeros_like(dd)

    row_means = np.mean(dd_norm, axis=1, keepdims=True)
    row_variances = np.std(dd_norm, axis=1, keepdims=True) + 1e-10
    anomaly_component = (dd_norm - row_means) / row_variances

    weights = 1.0 + alpha * (np.random.random(n) - 0.5) * intensity
    dd_preprocessed = anomaly_component * weights[:, np.newaxis]

    dd_preprocessed += np.random.normal(0, 1e-6, dd_preprocessed.shape)

    if original_shape:
        if original_shape[1] > m:
            padding = np.zeros((n, original_shape[1] - m))
            dd_preprocessed = np.hstack((dd_preprocessed, padding))
        elif original_shape[1] < m:
            dd_preprocessed = dd_preprocessed[:, :original_shape[1]]

    return weights, dd_preprocessed

def kan_univariate_layer(inputs, grid_size=5):
    """Simple Kolmogorov-Arnold univariate layer for feature mapping."""
    x = inputs.flatten()
    grid = np.linspace(x.min(), x.max(), grid_size)
    spline_coeffs = np.polyfit(x, x**2 + 0.1 * np.sin(x), 2) # Learnable univariate (spline approximation)
    mapped = np.polyval(spline_coeffs, x).reshape(inputs.shape)
    return mapped

def ev_anomaly_detection(dd, snr_global_light, sigma2, intensity):
    if not isinstance(dd, np.ndarray) or dd.ndim != 2:
        raise ValueError("Input dd must be a 2D NumPy array")
    n, m = dd.shape
    if n == 0 or m == 0:
        return np.zeros(n), np.zeros(n), 0.0

    row_means = np.mean(np.abs(dd), axis=1)
    data_std = np.std(dd) + 1e-10

    p_anomaly = np.zeros(n)
    for i in range(n):
        row_max = np.max(np.abs(dd[i, :]) + 1e-10)
        row_signal = dd[i, :] / (row_max + 1e-10) if row_max > 0 else np.zeros_like(dd[i, :])

        row_variance = np.std(dd[i, :]).reshape(1, -1)
        kan_feature = kan_univariate_layer(row_variance).flatten()[0] * 0.1

        dot_product = np.sum(row_signal * dd[i, :]) + kan_feature
        if np.allclose(dot_product, 0, atol=1e-8):
            logging.warning(f"Zero dot product detected for row {i}, adding perturbation.")
            dot_product += np.random.normal(0, data_std * 1e-4, 1)[0]

        norm = np.linalg.norm(dot_product) + 1e-10
        phase_angle = np.angle(dot_product / norm) if norm > 0 else 0
        phase_i = phase_angle * (3.0 + 3.0 * np.std(dd[i, :]) / (row_means[i] + 1e-10))

        anomaly_score = np.abs(phase_i) * snr_global_light / (sigma2 + 1e-10) * intensity
        if anomaly_score <= 0:
            anomaly_score = data_std * 1e-4
        p_anomaly[i] = np.clip(anomaly_score / (1 + anomaly_score), 0, 1)

    efficiency = np.mean(p_anomaly)

    anomaly_threshold = np.percentile(p_anomaly, 75)
    anomaly_map = (p_anomaly > anomaly_threshold).astype(int)

    return p_anomaly, anomaly_map, efficiency

def solve_ralf1_linear(dQ2, Q2, a, iterations=50):
    Np, Nf = dQ2.shape
    if Q2.shape[0] != Nf or a.shape[0] != Np:
        raise ValueError("Input shapes mismatch: dQ2 (Np, Nf), Q2 (Nf,), a (Np,)")

    A = np.zeros((Np, Np))
    b = np.zeros(Np)
    scale = 1e-3
    for k in range(Np):
        b[k] = np.sum(dQ2[k, :] * Q2) * scale
        for i in range(Np):
            A[k, i] = np.sum(dQ2[k, :] * dQ2[i, :]) * scale

    A += np.eye(Np) * 1e-2

    x = np.ones(Np) * 1e-6
    X_sum = np.zeros(Np)
    total_iterations = iterations

    for _ in range(iterations):
        indices = np.random.permutation(Np)
        x_iter = np.ones(Np) * 1e-6
        for i in indices:
            z = b[i] - np.dot(A[i, indices != i], x_iter[indices != i])
            A_ii = A[i, i] if A[i, i] != 0 else 1e-6
            x_iter[i] = z / A_ii
            x_iter[i] = np.clip(x_iter[i], -a[i], a[i])

```



```

        X_sum += x_iter

    x = X_sum / total_iterations
    return x

def filter_ralf1_matrix(source_matrix, intensity=1.0, iterations=50):
    Np, Nf = source_matrix.shape
    dQ2 = source_matrix
    Q2 = np.mean(source_matrix, axis=0)
    a = np.ones(Np) * np.max(np.abs(source_matrix))
    x = solve_ralf1_linear(dQ2, Q2, a, iterations)
    x_norm = x / (np.max(np.abs(x)) + 1e-10)
    x_norm = intensity * x_norm + (1 - intensity)
    filtered_matrix = source_matrix.copy()
    for i in range(Np):
        filtered_matrix[i, :] *= x_norm[i]
    return filtered_matrix

def hybrid_geo_filter(dd, intensity=0.1, iterations=15, alpha=0.5):
    if not isinstance(dd, np.ndarray) or dd.ndim != 2:
        raise ValueError("Input must be a 2D NumPy array")
    n, m = dd.shape
    if n == 0 or m == 0:
        return np.zeros_like(dd), np.zeros(n), np.zeros(m)

    dd_norm = dd / (np.max(np.abs(dd)) + 1e-10) * (10000.0 if n > 100 else 1000.0)

    weights_pre_light, dd_pre_light = hybrid_preprocess(dd_norm, intensity, alpha)

    if np.any(np.isnan(dd_pre_light)):
        logging.warning("NaN detected in dd_pre_light, replacing with zeros.")
        dd_pre_light = np.nan_to_num(dd_pre_light, nan=0.0)

    signal_norm = np.linalg.norm(dd_pre_light) + 1e-10
    noise_norm = np.linalg.norm(dd_pre_light - dd_norm) + 1e-10
    snr_global_light = 10 * np.log10(signal_norm**2 / noise_norm**2)
    sigma2 = np.var(dd_norm) + 1e-10
    p_anomaly, anomaly_map, efficiency = ev_anomaly_detection(dd_pre_light, snr_global_light, sigma2, intensity)

    if USE_KOLMOGOROV_ENHANCEMENTS and FAST_MODE:
        p_kolmo = fast_kolmogorov_ensemble(dd_pre_light)
        p_anomaly = np.maximum(p_anomaly, 0.4 * p_kolmo)

    snr_rows = np.zeros(n)
    for i in range(n):
        signal_power = np.linalg.norm(dd_pre_light[i, :])**2 + 1e-10
        noise_power = np.linalg.norm(dd_pre_light[i, :] - dd_norm[i, :])**2 + 1e-10
        snr_rows[i] = 10 * np.log10(signal_power / noise_power)
    snr_median = np.median(snr_rows)
    snr_threshold = max(np.percentile(snr_rows, 70.09), snr_median + 0.05)
    snr_anomaly_map = (snr_rows > snr_threshold).astype(int)
    p_combined = np.maximum(p_anomaly, snr_anomaly_map)

    filtered_matrix = np.zeros_like(dd_norm)
    high_prob_rows = np.where(p_combined >= 0.5)[0]
    if len(high_prob_rows) == 0:
        high_prob_rows = np.arange(n)
    if len(high_prob_rows) > 0:
        dd_high = dd_norm[high_prob_rows, :]
        weights_pre, dd_preprocessed = hybrid_preprocess(dd_high, intensity=intensity, alpha=alpha)
        logging.info(f"Rank of dd_preprocessed: {np.linalg.matrix_rank(dd_preprocessed)}/{len(high_prob_rows)}")
        filtered_matrix[high_prob_rows, :] = filter_ralf1_matrix(dd_preprocessed, intensity=intensity, iterations=iterations)
    else:
        weights_pre, dd_preprocessed = hybrid_preprocess(dd_norm[:, :], intensity, alpha=alpha)
        filtered_matrix[0, :] = filter_ralf1_matrix(dd_preprocessed, intensity=intensity, iterations=iterations)[0, :]

    low_prob_rows = np.where(p_combined < 0.5)[0]
    if len(low_prob_rows) > 0:
        filtered_matrix[low_prob_rows, :] = dd_norm[low_prob_rows, :] * 0.1

    signal_norm = np.linalg.norm(filtered_matrix) + 1e-10
    noise_norm = np.linalg.norm(filtered_matrix - dd_norm) + 1e-10
    snr_global = 10 * np.log10(signal_norm**2 / noise_norm**2)
    p_anomaly, anomaly_map, efficiency = ev_anomaly_detection(dd_preprocessed, snr_global, sigma2, intensity)

    # Extend p_anomaly to full size
    p_anomaly_full = np.zeros(n)
    p_anomaly_full[high_prob_rows] = p_anomaly
    if len(high_prob_rows) == 0:
        p_anomaly_full[0] = p_anomaly[0]

    snr_rows = np.zeros(n)
    for i in range(n):
        signal_power = np.linalg.norm(dd_pre_light[i, :])**2 + 1e-10
        noise_power = np.linalg.norm(dd_pre_light[i, :] - dd_norm[i, :])**2 + 1e-10
        snr_rows[i] = 10 * np.log10(signal_power / noise_power)
    snr_median = np.median(snr_rows)
    snr_threshold = max(np.percentile(snr_rows, 70.09), snr_median + 0.05)
    snr_anomaly_map = (snr_rows > snr_threshold).astype(int)
    p_combined = np.maximum(p_anomaly_full, snr_anomaly_map)

    return filtered_matrix, p_combined, snr_anomaly_map

def XFilter(dd0, key3=1):
    stdQ2=np.std(dd0)
    srdQ2=np.mean(dd0,axis=0)
    dd0=dd0-srdQ2
    dd, p_combined, snr_anomaly_map = hybrid_geo_filter(dd0, intensity=0.1, iterations=15, alpha=0.5)
    if key3==0:
        dd_=XFilterB(dd0)
        dd_, p_combined, snr_anomaly_map = hybrid_geo_filter(dd_, intensity=0.1, iterations=15, alpha=0.5)
        dd= hybrid_geo_filter(dd-dd_, intensity=0.1, iterations=15, alpha=0.5)[0]

    dd=(dd-np.mean(dd,axis=0))*stdQ2/np.std(dd)+srdQ2
    return dd # Return the filtered matrix

# === Faddeev Residual Refinement ===
def faddeev_residual_refinement(dQ4_B, dQ4_A):
    return hybrid_geo_filter(dQ4_B - dQ4_A, 0.15, 10, 0.6)[0]

# =====
# END OF CODE
# =====

import psutil
def CheckTemp(aTemp=MaxTemp):
    battery = psutil.sensors_battery()
    if not battery is None:
        return int(battery.power_plugged)
    return 1

DETERM=0.1

DETERM=0.1

def RandomQ(Nfx, NQRandm_=0):
    while not CheckTemp():
        tm.sleep(60)
    QRandm=np.asarray(range(512),float)
    NQRandm=0
    KK=3e6

```



```

l1iX=np.zeros(Nfx,float)
pp=0
while pp<0.7:
    for ii in range(3):
        try:
            z=(QRandm.[NQRandm+1])/KK
            atim0=tm.time()
            tm.sleep(z)
            atim=tm.time()
            dd=int(((atim-atim0)/z-1)/1000)
            zz=np.asarray(sample(list(range(Nfx)),Nfx),float)/KK
            LFib1340(dd).shuffle(zz)
            LFib1340(int(2*dd/(1+np.sqrt(5))))).shuffle(QRandm_)

            if NQRandm>0:
                l1iX=l1iX+zz
                NQRandm=NQRandm+1
        except:
            NQRandm=0

    k2, pp = scp.skewtest(l1iX)

rr2=[[],[]]
rr2[0]= l1iX.copy()
rr2[1]= np.asarray(np.linspace(0,Nfx-1,Nfx),int)
m=[rr2[j][l] for j in range(len(rr2))] for l in range(len(rr2[0]))]
m.sort(key=itemgetter(0))
rr2=[m[j][l] for j in range(len(m))] for l in range(len(m[0]))]
l1iXX=np.asarray(rr2[1],int)
return l1iXX

def filterFourierQ(arxx,arb,NNew,NChan,key=-1,key2=0):
    if key2==-2:
        arxx=np.concatenate((arxx, arxx[::-1]))
        arb=np.concatenate((arb, arb[::-1]))
        Nnl=2*NNew
    else:
        Nnl=NNew
        Nfl=int(len(arb)/NChan)
        Nfl_=int(len(arxx)/NChan)

        #ar_=np.zeros(Nnl,float)
        farx=np.zeros(8*Nnl,float)-1e32

        az=int(np.floor(Nfl/Nnl))

        gg0=0
        for l in range(NChan):
            for i in range(az):
                ar =arb[Nfl-(az-i)*Nnl+Nfl*l:Nfl+Nnl-(az-i)*Nnl+Nfl*l].copy()
                gg0=gg0+np.sum(ar_*ar_)
                #ar_=ar_[:-1].copy()
                ar =ar_-ar_[0]
                #ar_=ar_-np.mean(ar_)
                ar_x=ar_[:-1].copy()
                ar_=np.concatenate((ar_x,-2*((key<0)-.5)*ar_))
                ar_=np.concatenate((ar_[:-1],ar_))
                ar_=ar_-ar_[len(ar_)-1]
                ar_=np.concatenate((ar_,-ar_))
                ar__=abs(np.fft.fft(ar_))
                farx=np.maximum(farx,ar__)
            #gg0=np.sqrt(gg0)/(NChan*az*Nnl)

        gg=0
        arxr=arxx.copy()
        for l in range(NChan):
            ar =arxx[Nfl*(l+1)-Nnl:Nfl*(l+1)].copy()
            #par_= savgol_filter(ar_, 14, 5)
            par_=ar_.copy()
            #ar_=ar_-ar_[0]
            ar_=ar_[:-1].copy()
            ar_=ar_-par_[0]
            #ar_=ar_-ar_[len(ar_)-1]
            #ar_=ar_-np.mean(ar_)
            ar_x=ar_[:-1].copy()
            ar_=np.concatenate((ar_,-2*((key<0)-.5)*ar_x))
            ar_=np.concatenate((ar_[:-1],ar_))
            ar_=ar_-ar_[len(ar_)-1]
            ar_=np.concatenate((ar_,-ar_))
            farxx=np.fft.fft(ar_)
            mfarxx=np.abs(farxx)+1e-32
            farxxx=np.zeros(8*Nnl,complex)

            A1=np.exp(DETERM*np.std(np.log(mfarxx[5:8*Nnl-4]))+np.mean(np.log(mfarxx[5:8*Nnl-4])))

            for j in range(1,8*Nnl):
                if mfarxx[j]>A1:
                    farxxx[j]=(farxx[j]/mfarxx[j])*np.sqrt(farx[j]*mfarxx[j])
                    #farxxx[j]=(farxx[j]/mfarxx[j])*farx[j]

            farxxx[0]=0*farxx[0]

            aaa=np.fft.ifft(farxxx).real
            aaa=aaa[3*Nnl:5*Nnl].copy()
            aaa=-(aaa[0:Nnl]-2*((key<0)-.5)*aaa[2*Nnl:Nnl-1:-1])/2)#aaa[0:Nnl][::-1]#
            arxr[Nfl-Nnl+Nfl*1:Nfl+Nfl*1]=aaa.copy()
            #arxr[Nfl-Nnl+Nfl*1:Nfl+Nfl*1]= savgol_filter(arxr[Nfl-Nnl+Nfl*1:Nfl+Nfl*1], 14, 5)
            gg=gg-arxr[Nfl-Nnl+Nfl*1]+arb[Nfl-Nnl+Nfl*1-1]

        gg=gg/NChan
    if key2==-2:
        aln=int(len(arxr)/2)
        arxr=arxr[:aln]
    return arxr

def RALFICalculation2(arr_bx,arr_c,Nf,NNew,NNew0,NChan,Nhh,iProc,Nproc):
    warnings.filterwarnings("ignore", category=RuntimeWarning)
    sz=Nf*NChan
    MM=3
    Nzz=Nhh
    Nhh=Nzz

    Ndel=MM
    NCh=int(np.ceil(sz/Ndel))
    Ndel0=MM
    NCh0=int(np.ceil(sz/Ndel0))

    arr_b=np.asarray(arr_bx,float)

    hh=0
    hh_=0
    Nzz0=Nzz
    Atim_0=tm.time()
    dd1_x=[]
    dd2_x=[]
    while hh<Nhh:
        if hh==0:
            AMX=np.zeros((Nhh,sz),float)
            AMN=np.zeros((Nhh,sz),float)
            dd10=np.zeros((Nhh+1,sz),float)
            dd20=np.zeros((Nhh+1,sz),float)
            max_dd1=np.zeros((Nhh+1,sz),float)-1e32
            min_dd2=np.zeros((Nhh+1,sz),float)+1e32

```



```

arr_bbbxx1=np.zeros((Nhh,sz),float)
arr_bbbxx2=np.zeros((Nhh,sz),float)

Nzz=Nzz0
rr2=np.zeros((Nhh+1,sz),float)
rr2[hh]=np.asarray(arr_b,float)
if hh==1:
    Nzz=int(Nzz0/2+1)

www=0
while not www:
    try:
        arrrx=hkl.load("ralfrez.rlf2")
        www=1
    except:
        tm.sleep(0.1)

if len(arrrx)>=Nproc:
    return rr2[hh]/0

liix=np.zeros((sz,sz),int)
dQ3=np.zeros((sz,sz),float)
mDD=np.zeros((sz,sz),float)

aa=RandomQ(sz)
liiB=np.concatenate((aa,aa,aa))
aa=RandomQ(sz)
liiD=np.concatenate((aa,aa,aa))

rrr=rr2[hh].copy()
# rrr=rrr[0]
# rrr[1:]=np.diff(rrr)

for i in range(sz):
    liix[i]=liiB[liiD[i]:sz+liiD[i]].copy()#range(sz)#
    dQ3[i]=rrr[liix[i]].copy()

astart=np.inf
dQ3=dQ3.reshape((sz*sz))
astart=dQ3[0]
dQ3[1:]=np.diff(dQ3)
dQ3[0]=0
dQ3=dQ3.reshape((sz,sz))
dQ3_=dQ3.copy()

D=np.std(dQ3)

R4=np.ones(sz,float)
for l in range(NChan):
    R4[Nf-NNew+Nf*L:Nf+Nf*L]=0

for i in range(sz):
    mDD[i]=R4[liix[i]].copy()

seqq=dQ3.reshape(sz*sz))[1:]*np.ceil(0.5*(1/(mDD.reshape(sz*sz)==1)[0:sz*sz-1]+1/(mDD.reshape(sz*sz)==1)[1:]))
seqq_=mDD.reshape(sz*sz)*0
seqq_[0]=1
seqq_[1:]=(abs(seqq)==np.inf)+np.isnan(seqq)
seqq=seqq_.reshape((sz,sz))
seqq=np.asarray(list(filter(Lambda x: abs(x)!= np.inf, seqq)),float)
seqq=np.asarray(list(filter(Lambda x: abs(np.isnan(x))!= 1, seqq)),float)

#####
aa=RandomQ(sz)
NumFri0=np.concatenate((aa, aa, aa))
aa=RandomQ(sz)
NumFri0_=np.concatenate((aa, aa, aa))
aa=RandomQ(sz)
rR0=np.concatenate((aa, aa, aa))
aa=RandomQ(sz)
liiC=np.concatenate((aa, aa, aa))
aa=RandomQ(sz)
aa=aa-np.mean(aa)
r5=aa.copy()
r5=3*r5*D/np.std(r5)
r5=r5-np.mean(r5)
r5=np.concatenate((r5, r5))
aa=RandomQ(sz)
ss4=np.concatenate((aa, aa, aa))
zz=0
WW=0

while zz<Nzz and WW>=-2*Nhh:
    KY=1
    # dQ3=dQ3_.copy()
    dQ3mx=np.zeros((sz,sz),np.float16)-1e3
    dQ3mn=np.zeros((sz,sz),np.float16)+1e3
    dQ3num=np.zeros((sz,sz),int)
    NumFri=NumFri0_[NumFri0[ss4[zz]]:NumFri0[ss4[zz]]+2*sz].copy()
    NumFri_=NumFri0[NumFri0_[ss4[zz]]:NumFri0_[ss4[zz]]+2*sz].copy()
    for kkk in range(Nzz):
        aa=ss4[0]
        ss4[0:len(ss4)-1]=ss4[1:].copy()
        ss4[len(ss4)-1]=aa
        rR=rR0[liiC[ss4[zz]]:liiC[ss4[zz]]+2*sz].copy()
        rR_=rR0[liiC[ss4[len(ss4)-2*sz+zz]]:liiC[ss4[len(ss4)-2*sz+zz]]+2*sz].copy()

        kk=-1
        xxx=0
        while kk <Ndel-1 and xxx==0 and np.sum(dQ3num==0)>0:
            kk=kk+1
            ii=int(kk*NCh)
            k=-1
            while k<Ndel0-1 and xxx==0:
                k=k+1
                i=int(k*NCh0)
                dQ4=np.zeros((NCh,NCh0),float)
                mDD4=np.zeros((NCh,NCh0),float)
                mDD4_=np.zeros((NCh,NCh0),float)
                mDD4_A=np.zeros((NCh,NCh0),float)
                mDD4_B=np.zeros((NCh,NCh0),float)
                for ll in range(NCh0):
                    dQ4[:,ll]=dQ3[NumFri[ii:ii+NCh],NumFri_[i+ll]].copy()
                    mDD4[:,ll]=mDD[NumFri[ii:ii+NCh],NumFri_[i+ll]].copy()

                dQ4=dQ4-np.mean(dQ4)
                P=np.zeros(3,float)

                for ll in range(NCh0):
                    vvv=r5[rR[ss4[ll]+zz]:rR[ss4[ll]+zz]+NCh]*(1-mDD4[:,ll])
                    mDD4_A[:,ll]=vvv.copy()
                    mDD4_B[:,ll]=vvv.copy()#vvv[::-1].copy()

                mDD4_B=mDD4_A.copy()#*0
                # KY=KY*(-1)
                mDD4_A=KY*mDD4_A*(mDD4_A>0)
                mDD4_B=KY*mDD4_B*(mDD4_B>0)

                nNxA=sum(sum(mDD4==1))
                nNxA_=sum(sum(mDD4==0))
                # if nNxA>nNxA_ and nNxA_>0:
                if nNxA_>0:
                    seqA0=(dQ4.reshape(NCh*NCh0))[1:]*np.ceil(0.5*(1/(mDD4.reshape(NCh*NCh0)==1)[0:NCh*NCh0-1]+1/(mDD4.reshape(NCh*NCh0)==1)[1:]))

```



```

seqA0_ = mDD4.reshape(NCh*NCh0)*0
seqA0_[0]=1
seqA0_[1:]=(abs(seqA0)==np.inf)+np.isnan(seqA0)
seqA0_ =seqA0_.reshape((NCh,NCh0))
seqA0=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqA0)),float)
seqA0=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqA0)),float)

seqA=(dQ4.reshape(NCh*NCh0))[1:]*np.ceil(0.5*np.fabs(1/(1*(mDD4.reshape(NCh*NCh0)==1)[0:NCh*NCh0-1]-1*(mDD4.reshape(NCh*NCh0)==1)[1:])))
seqA_ = mDD4.reshape(NCh*NCh0)*0
seqA_[0]=1
seqA_[1:]=(abs(seqA)==np.inf)+np.isnan(seqA)
seqA_ =seqA_.reshape((NCh,NCh0))
seqA=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqA)),float)
seqA=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqA)),float)

dQ4_B= -XFilter(mDD4_A+dQ4)-((mDD4_A))#,0)
dQ4_A= -XFilter(mDD4_B-dQ4)+((mDD4_B))#,0)

dQ4_B=(dQ4_B-dQ4_A)/2
# asr1=abs(dQ4_B-xxxx)>abs(dQ4_A-xxxx)
# asr2=abs(dQ4_B-xxxx)<abs(dQ4_A-xxxx)
# dQ4_B=dQ4_B*asr1+dQ4_A*asr2+(dQ4_B+dQ4_A)*(asr1==asr2)/2

# dQ4_B = faddeev_residual_refinement(dQ4_B, dQ4_A)
# # # dQ4_B=dQ4_B-dQ4_A
dQ4_A=dQ4_B.copy()

seqB=((dQ4_A).reshape(NCh*NCh0))[1:]*np.ceil(0.5*np.fabs(1/(1*(mDD4.reshape(NCh*NCh0)==1)[0:NCh*NCh0-1]-1*(mDD4.reshape(NCh*NCh0)==1)[1:])))
seqB=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqB)),float)
seqB=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqB)),float)
seqC=((dQ4_B).reshape(NCh*NCh0))[1:]*np.ceil(0.5*np.fabs(1/(1*(mDD4.reshape(NCh*NCh0)==1)[0:NCh*NCh0-1]-1*(mDD4.reshape(NCh*NCh0)==1)[1:])))
seqC=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqC)),float)
seqC=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqC)),float)
try:
    P_1=P.copy()
    P_2=P.copy()
    P_1[0:2]=np.polyfit(seqA,seqB,1)
    P_2[0:2]=np.polyfit(seqA,seqC,1)
    # print(100*scp.pearsonr(seqA,seqB)[0])
    P_1[0]=np.std(seqB)/np.std(seqA)
    P_1[1]=np.mean(seqB)-P_1[0]*np.mean(seqA)
    P_1[2]=0
    P_2[0]=np.std(seqC)/np.std(seqA)
    P_2[1]=np.mean(seqC)-P_2[0]*np.mean(seqA)
    P_2[2]=0
    if P_1[0]>0 and P_2[0]>0:## and 100*scp.pearsonr(seqA,seqB)[0]>0 and 100*scp.pearsonr(seqA,seqC)[0]>0:## and not (abs(P_1[0]-1)>0.5 or abs(P_2[0]-1)>0.5):
        dQ4_A=(dQ4_A-P_1[1])/P_1[0]
        dQ4_B=(dQ4_B-P_2[1])/P_2[0]
        for ll in range(NCh0):
            seqA=dQ3_[NumFri[ii:ii+NCh],NumFri_ [i+ll]]+seqA0_[,ll]*np.maximum(dQ3mx[NumFri[ii:ii+NCh],NumFri_ [i+ll]]-dQ3_[NumFri[ii:ii+NCh],NumFri_ [i+ll]],dQ4_A[:,ll]-dQ3_[NumFri[ii:ii+NCh],NumFri_ [i+ll]])
            seqB=dQ3_[NumFri[ii:ii+NCh],NumFri_ [i+ll]]+seqA0_[,ll]*np.minimum(dQ3mn[NumFri[ii:ii+NCh],NumFri_ [i+ll]]-dQ3_[NumFri[ii:ii+NCh],NumFri_ [i+ll]],dQ4_B[:,ll]-dQ3_[NumFri[ii:ii+NCh],NumFri_ [i+ll]])
            ab=dQ3num[NumFri[ii:ii+NCh],NumFri_ [i+ll]]*np.asarray(dQ3mx[NumFri[ii:ii+NCh],NumFri_ [i+ll]],np.float16)
            ba=dQ3num[NumFri[ii:ii+NCh],NumFri_ [i+ll]]*np.asarray(dQ3mn[NumFri[ii:ii+NCh],NumFri_ [i+ll]],np.float16)
            dQ3mx[NumFri[ii:ii+NCh],NumFri_ [i+ll]]=(ab+seqA)/(dQ3num[NumFri[ii:ii+NCh],NumFri_ [i+ll]]+1)
            dQ3mn[NumFri[ii:ii+NCh],NumFri_ [i+ll]]=(ba+seqB)/(dQ3num[NumFri[ii:ii+NCh],NumFri_ [i+ll]]+1)

            dQ3num[NumFri[ii:ii+NCh],NumFri_ [i+ll]]=dQ3num[NumFri[ii:ii+NCh],NumFri_ [i+ll]]+1
            dQ3[NumFri[ii:ii+NCh],NumFri_ [i+ll]]=(dQ3mx[NumFri[ii:ii+NCh],NumFri_ [i+ll]]+dQ3mn[NumFri[ii:ii+NCh],NumFri_ [i+ll]])/2

        ll=ll+1
    # else:
    #     xxx=1
except:
    xxx=1
else:
    xxx=1

#dQ3=dQ3_.copy()
dQ3mxL=dQ3mx.copy()
dQ3mnL=dQ3mn.copy()

if np.isnan(np.std(dQ3mx)) or np.isnan(np.std(dQ3mn)):
    xxx=1
if xxx==0:
    if zz==0:
        AsrXMx=dQ3mx.copy()
        AsrXMn=dQ3mn.copy()
    else:
        # AsrXMx=(AsrXMx*zz+dQ3mx)/(zz+1)
        # AsrXMn=(AsrXMn*zz+dQ3mn)/(zz+1)
        AsrXMx=np.maximum(AsrXMx,dQ3mx)
        AsrXMn=np.minimum(AsrXMn,dQ3mn)
        # AsrXMx=(AsrXMx*zz+np.maximum(AsrXMx,dQ3mx))/(zz+1)
        # AsrXMn=(AsrXMn*zz+np.minimum(AsrXMn,dQ3mn))/(zz+1)

        dQ3=(AsrXMx+AsrXMn)/2
        AsrXMx_ =AsrXMx.copy()
        AsrXMn_ =AsrXMn.copy()

        WW=0
        ZZ=ZZ+1
    else:
        aa=ss4[0]
        ss4[0:len(ss4)-1]=ss4[1:].copy()
        ss4[len(ss4)-1]=aa
        WW=WW-1

wwww=0
while not wwww:
    try:
        arrrx=hkl.load("ralfrez.rlf2")
        wwww=1
    except:
        tm.sleep(0.1)

if len(arrrx)>=Nproc:
    return rr2[hh]/0

hh0=hh
if not WW<0:
    l==1
    #AsrXMx_ =filter_ralf1_matrix(AsrXMx_, intensity=1.0, iterations=100)
    seqq_1=((AsrXMx_).reshape(sZ*sZ))[1:]*np.ceil(0.5*(1/(mDD.reshape(sZ*sZ)==1)[0:sZ*sZ-1]+1/(mDD.reshape(sZ*sZ)==1)[1:])))
    seqq_1=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqq_1)),float)
    seqq_1=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqq_1)),float)

    #AsrXMn_ =filter_ralf1_matrix(AsrXMn_, intensity=1.0, iterations=100)
    seqq_2=((AsrXMn_).reshape(sZ*sZ))[1:]*np.ceil(0.5*(1/(mDD.reshape(sZ*sZ)==1)[0:sZ*sZ-1]+1/(mDD.reshape(sZ*sZ)==1)[1:])))
    seqq_2=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqq_2)),float)
    seqq_2=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqq_2)),float)
    P_1[0:2]=np.polyfit(seqq,seqq_1,1)
    P_2[0:2]=np.polyfit(seqq,seqq_2,1)
    if 100*scp.pearsonr(seqq,seqq_1)[0]>0 and 100*scp.pearsonr(seqq,seqq_2)[0]>0 and P_1[0]>0 and P_2[0]>0:##not abs(P[0]-1)>0.5 and :
        l==1
        AsrXMx_=(AsrXMx_-P_1[1])/P_1[0]
        AsrXMn_=(AsrXMn_-P_2[1])/P_2[0]
    else:
        WW=-1
if not WW<0:
    dd0=0
    asr1=abs(AsrXMx_-dd0)>abs(AsrXMn_-dd0)

```



```

asr2=abs(AsrXMx_-dd0)<abs(AsrXMn_-dd0)
AsrXMx_=AsrXMx_*(asr1+AsrXMn_*(asr2+(AsrXMx_+AsrXMn_)*(asr1==asr2)/2
AsrXMn_=AsrXMx_.copy()

if not astart==np.inf:
    dd1=AsrXMx_.reshape((sz*sz))
    dd1=astart+np.cumsum(dd1)
    AsrXMx_=dd1.reshape((sz,sz))
    dd1=AsrXMn_.reshape((sz*sz))
    dd1=astart+np.cumsum(dd1)
    AsrXMn_=dd1.reshape((sz,sz))

AsrXMx_=(AsrXMx_+AsrXMn_)/2
AsrXMn_=AsrXMx_.copy()

aMx_=0
aMn_=0
aMx=np.zeros(sz,float)-1e32
aMn=np.zeros(sz,float)+1e32
for i in range(sz):
    aMx[liix[i]]=np.maximum(aMx[liix[i]],AsrXMx_[i])
    aMn[liix[i]]=np.minimum(aMn[liix[i]],AsrXMn_[i])
    # aMx_=(aMx_*i+aMx)/(i+1)#
    # aMn_=(aMn_*i+aMn)/(i+1)#
aMx_=aMx_.copy()
aMn_=aMn_.copy()
ann=sum(np.isnan(aMx_+aMn_))
if ann==0:
    if hh==0:
        AMX[hh]=aMx_.copy()
        AMN[hh]=aMn_.copy()
        arr_bbbxxx1[hh]=AMX[hh].copy()
        arr_bbbxxx2[hh]=AMN[hh].copy()
        D1=10
    else:
        AMX[hh]=aMx_.copy()
        AMN[hh]=aMn_.copy()
        arr_bbbxxx1[hh]=(arr_bbbxxx1[hh-1]*(hh-1)+AMX[hh])/(hh)
        arr_bbbxxx2[hh]=(arr_bbbxxx2[hh-1]*(hh-1)+AMN[hh])/(hh)
        dd1=[]
        dd2=[]
        D1=[]
        D2=[]
        for l in range(NChan):
            dd1_.append(arr_bbbxxx1[hh-1][Nf-NNew+Nf*L:Nf+Nf*L].copy())
            dd2_.append(arr_bbbxxx2[hh-1][Nf-NNew+Nf*L:Nf+Nf*L].copy())
            D1_.append(arr_bbbxxx1[hh]-arr_bbbxxx1[hh-1][Nf-NNew+Nf*L:Nf+Nf*L].copy())
            D2_.append(arr_bbbxxx2[hh]-arr_bbbxxx2[hh-1][Nf-NNew+Nf*L:Nf+Nf*L].copy())
        D1=np.asarray(D1_,float)
        D2=np.asarray(D2_,float)
        dd1=np.asarray(dd1_,float)
        dd2=np.asarray(dd2_,float)
        D1=np.std(D1_+D2_)/np.std(dd1+dd2)*np.sqrt(hh+1)/np.sqrt(1+(hh+1))

ann=1
try:
    for l in range(NChan):
        rrr[Nf*L:Nf+Nf*L]=savgol_filter(rrr[Nf*L:Nf+Nf*L],14,5)

dd1=(filterFourierQ(AMX[hh],rrr,NNew,NChan))
dd2=(filterFourierQ(AMN[hh],rrr,NNew,NChan))
dd0=(dd1+dd2)/2
asr1=abs(dd1-dd0)>abs(dd2-dd0)
asr2=abs(dd1-dd0)<abs(dd2-dd0)
dd1=dd1*asr1+dd2*asr2+(dd1+dd2)*(asr1==asr2)/2
dd2=dd1.copy()
if sum(np.abs(dd1+dd2))==np.inf==0 and D1>DETERM:
    sr2_1=[]
    sr2_2=[]
    sarr_c=[]
    for l in range(NChan):
        sr2_1.append((dd1)[Nf-NNew+int(0.3*(NNew-NNew0))+Nf*L:Nf-NNew0+Nf*L])
        sr2_2.append((dd2)[Nf-NNew+int(0.3*(NNew-NNew0))+Nf*L:Nf-NNew0+Nf*L])
        sarr_c.append(sarr_c[(NNew-NNew0)*l+int(0.3*(NNew-NNew0)):(NNew-NNew0+(NNew-NNew0)*l).copy()])
    sr2_1=np.asarray(sr2_1,float)
    sr2_2=np.asarray(sr2_2,float)
    sarr_c=np.asarray(sarr_c,float)
    sr2_1=sr2_1.reshape((len(sr2_1)*len(sr2_1[0]))))
    sr2_2=sr2_2.reshape((len(sr2_2)*len(sr2_2[0]))))
    sarr_c=sarr_c.reshape((len(sarr_c)*len(sarr_c[0]))))

    P_1=P.copy()
    P_2=P.copy()

    # sr2_1=(sr2_1+sr2_2)/2
    # sr2_2=sr2_1.copy()
    P_1[0:2]=np.polyfit(sarr_c,sr2_1,1)
    P_2[0:2]=np.polyfit(sarr_c,sr2_2,1)

    P_1[0]=np.std(sr2_1)/np.std(sarr_c)
    P_1[1]=np.mean(sr2_1)-P_1[0]*np.mean(sarr_c)
    P_2[0]=np.std(sr2_2)/np.std(sarr_c)
    P_2[1]=np.mean(sr2_2)-P_2[0]*np.mean(sarr_c)

#abs(P_1[0]-P_2[0])/(abs(P_1[0])+abs(P_2[0]))<0.2
if abs(P_1[0]-P_2[0])/(abs(P_1[0])+abs(P_2[0]))<1 and P_1[0]>0 and P_2[0]>0:# and 100*scp.pearsonr(sarr_c,((sr2_1-P_1[1])/P_1[0]+(sr2_2-P_2[1])/P_2[0]))[0]>0:
    for l in range(NChan):
        dd1[Nf-NNew+Nf*L:Nf+Nf*L]=[dd1[Nf-NNew+Nf*L:Nf+Nf*L]-P_1[1])/P_1[0]
        dd2[Nf-NNew+Nf*L:Nf+Nf*L]=[dd2[Nf-NNew+Nf*L:Nf+Nf*L]-P_2[1])/P_2[0]
        max_dd1[hh]=rr2[hh].copy()
        min_dd2[hh]=rr2[hh].copy()
        for l in range(NChan):
            if hh==0:
                max_dd1[hh,Nf-NNew+Nf*L:Nf+Nf*L]=dd1[Nf-NNew+Nf*L:Nf+Nf*L].copy()
                min_dd2[hh,Nf-NNew+Nf*L:Nf+Nf*L]=dd2[Nf-NNew+Nf*L:Nf+Nf*L].copy()
            else:
                max_dd1[hh,Nf-NNew+Nf*L:Nf+Nf*L]=np.maximum(max_dd1[hh-1,Nf-NNew+Nf*L:Nf+Nf*L],(dd1[Nf-NNew+Nf*L:Nf+Nf*L]))
                min_dd2[hh,Nf-NNew+Nf*L:Nf+Nf*L]=np.minimum(min_dd2[hh-1,Nf-NNew+Nf*L:Nf+Nf*L],(dd2[Nf-NNew+Nf*L:Nf+Nf*L]))

        hh=hh+1
        ann=0
        # rr2[hh]=(max_dd1[hh-1]+min_dd2[hh-1])/2

        dd1=max_dd1[hh-1].copy()
        dd2=min_dd2[hh-1].copy()
        dd0=(dd1+dd2)/2
        asr1=abs(dd1-dd0)>abs(dd2-dd0)
        asr2=abs(dd1-dd0)<abs(dd2-dd0)
        rr2[hh]=dd1*asr1+dd2*asr2+(dd1+dd2)*(asr1==asr2)/2
        rr2[hh]=(rr2[hh-1]*(hh-1)+filterFourierQ(rr2[hh],rrr,NNew,NChan))/hh

##rr2[hh]=(rr2[hh-1]*(hh-1)+rr2[hh])/hh
sr2=[]
sarr_c=[]
for l in range(NChan):
    sr2.append(rr2[hh,Nf-NNew+int(0.3*(NNew-NNew0))+Nf*L:Nf-NNew0+Nf*L].copy())
    sarr_c.append(sarr_c[(NNew-NNew0)*l+int(0.3*(NNew-NNew0)):(NNew-NNew0+(NNew-NNew0)*l).copy()])
sr2=np.asarray(sr2,float)
sarr_c=np.asarray(sarr_c,float)
sr2=sr2.reshape((len(sr2)*len(sr2[0]))))
sarr_c=sarr_c.reshape((len(sarr_c)*len(sarr_c[0]))))

```



```

P[0:2]=np.polyfit(sarr_c,sr2,1)
P[0]=np.std(sr2)/np.std(sarr_c)
P[1]=np.mean(sr2)-P[0]*np.mean(sarr_c)

if P[0]>0:# and 100*scp.pearsonr(sarr_c,sr2)[0]>0:#and :
    dd1_x=[]
    dd2_x=[]
    for l in range(NChan):
        rr2[hh,Nf-NNew+Nf*1:Nf+Nf*1]=(rr2[hh,Nf-NNew+Nf*1:Nf+Nf*1]-P[1])/P[0]
        rr2[hh,Nf*1:Nf-NNew+Nf*1]=arr_b[Nf*1:Nf-NNew+Nf*1].copy()
        rr2[hh,:]= savgol_filter(rr2[hh,:], 14, 5)
    #P[0:2]=np.polyfit(sarr_c,sr2,1)

    #D1=0.2
    if hh==Nhh:
        if abs(P[0]-1.)<0.5 and sum(abs(rr2[hh])==np.inf)==0 and D1<1:
            anamef="fraf.f.tmp"
            fo = open(anamef, "w")
            Atim_1=tm.time()
            fo.write(str(iProc)+' it=%d, K=%3.2f Tm=%5.2f\n'%(hh,D1,Atim_1-Atim_0))
            fo.close()
            return np.asarray(rr2[hh],float)
        else:
            hh=hh0
    else:
        hh=hh0
    else:
        hh=hh0

except:
    hh=hh0

if hh0==hh:
    hh=hh+1
    if hh>1:
        hh=hh-2
    else:
        hh=0

    if hh_>2*Nhh:
        return rr2[hh]/0

def RALF1FiltrQ(*args):
    #####
    ww=0
    while (not ww):
        try:
            with open("filtrx.tmp", "rb") as file:
                args=file.read()
                file.close()
                ww=1
        except:
            tm.sleep(0.1)
    [NChan, Nf, NNew, Nhh, Nproc]=np.asarray(struct.unpack('5i', args[0:20]),np.int32)
    anamef="maxmin_.xxx"
    fo = open(anamef, "w")
    fo.write('%s'%(Nproc)+'\n')
    fo.close()
    arr_bb=np.asarray(struct.unpack('%sf'%(Nf), args[20:(4*Nf+20)]),np.float32)
    #####

    NNew0=int(NNew*1.2)

    #Nhh=1
    Nhh0=Nhh
    astar0=np.inf
    # astar0=arr_bb[0]

    # arr_bb[0]=0

    arr_b=np.asarray(arr_bb,float)

    arr_c=[]
    aa=RandomQ(NNew0)
    aa=aa-np.mean(aa)
    #aa= savgol_filter(aa, 14, 5)
    aa=9*aa/np.std(aa)*np.std(arr_b)
    ss4=np.concatenate((aa, aa, aa, aa)) #*0
    for l in range(NChan):
        arr_b[Nf-NNew0+Nf*1:Nf+Nf*1]=(astar0==np.inf)*arr_b[Nf-NNew0+Nf*1-1]+ss4[l:NNew0+l]
        arr_b[Nf*1:Nf+Nf*1]=np.cumsum(arr_b[Nf*1:Nf+Nf*1])
        arr_bb[Nf*1:Nf+Nf*1]=np.cumsum(arr_bb[Nf*1:Nf+Nf*1])
        arr_c.append(arr_b[Nf-NNew0+Nf*1:Nf-NNew+Nf*1].copy())

    arr_c=np.asarray(arr_c,float)
    arr_c=arr_c.reshape((len(arr_c)*len(arr_c[0])))

    while l==1:
        hh=0
        ann=0
        arr_bbx=np.zeros((Nhh*2,Nf),float)
        Koef=np.zeros(Nhh0+1,float)
        KoefA=np.zeros(Nhh0+1,float)
        NumIt=3*(int((Nhh0)/2)+1)

        while hh<Nhh:
            if hh<Nhh:
                arr_bbbxxx=RALF1Calculation2(arr_b,arr_c,Nf,NNew0,NNew,NChan,NumIt,args[0],Nproc)

                wwwww=0
                while not wwwww:
                    try:
                        arrrrx=hkl.load("ralfrez.rlf2")
                        wwwww=1
                    except:
                        tm.sleep(0.1)

                if len(arrrrx)>=Nproc:
                    return arr_bbbxxx/0
                if (sum(np.abs(arr_bbbxxx)==np.inf)==0 and sum(np.isnan(arr_bbbxxx))==0):
                    Nf=NNew0+int(NNew0*1.2)
                    if Nf >=Nf:
                        Nf=Nf-1
                    arr_bbbxxx=np.zeros(Nf_*NChan,float)
                    arr_c=[]
                    aa=RandomQ(NNew0)
                    aa=aa-np.mean(aa)
                    aa=3*aa/np.std(aa)*np.std(arr_bbbxxx)
                    ss4=np.concatenate((aa, aa, aa, aa))
                    for l in range(NChan):
                        arr_bbbxxx[Nf_*1:Nf+Nf_*1]=np.asarray(arr_bbbxxx[Nf*(l+1):Nf-Nf_-1+Nf*1:-1],float)
                        arr_c.append(arr_bbbxxx[Nf_-NNew0+Nf_*1:Nf_-Nf_-NNew+Nf_*1].copy())
                        arr_bbbxxx[Nf_-NNew0+Nf_*1:Nf_+Nf_*1]=(astar0==np.inf)*arr_bbbxxx[Nf_-NNew0+Nf_*1-1]+ss4[l:NNew0+l]
                    arr_c=np.asarray(arr_c,float)
                    arr_c=arr_c.reshape((len(arr_c)*len(arr_c[0])))
                    arr_bbbxxx_y=RALF1Calculation2(arr_bbbxxx,arr_c,Nf_,NNew0,NNew,NChan,NumIt,args[0],Nproc)
                    if (sum(np.abs(arr_bbbxxx_y)==np.inf)==0 and sum(np.isnan(arr_bbbxxx_y))==0):
                        for l in range(NChan):
                            if not astar0==np.inf:
                                if l==0:
                                    mm1=np.cumsum(arr_bb[Nf-NNew0-(Nf_-NNew0)+Nf*1:Nf*(l+1)-(Nf_-NNew0)])
                                    mm2=np.cumsum(arr_bbbxxx_y[Nf_*(l+1):Nf_-NNew0-1+Nf_*1:-1])
                                else:

```



```

mm1=np.concatenate((mm1,np.cumsum(arr_bb[Nf-NNew0-(Nf_-NNew0)+Nf*l:Nf*(l+1)-(Nf_-NNew0)])))
mm2=np.concatenate((mm2,np.cumsum(arr_bbbxxx_y[Nf_*(l+1):Nf_-NNew0-1+Nf_*l:-1])))

else:
    if l==0:
        mm1=arr_bb[Nf-NNew0-(Nf_-NNew0)+Nf*l:Nf*(l+1)-(Nf_-NNew0)].copy()
        mm2=arr_bbbxxx_y[Nf_*(l+1):Nf_-NNew0-1+Nf_*l:-1].Copy()
    else:
        mm1=np.concatenate((mm1,arr_bb[Nf-NNew0-(Nf_-NNew0)+Nf*l:Nf*(l+1)-(Nf_-NNew0)].copy()))
        mm2=np.concatenate((mm2,arr_bbbxxx_y[Nf_*(l+1):Nf_-NNew0-1+Nf_*l:-1].copy()))

ann=(sum(np.abs(mm1)==np.inf)>0 + sum(np.isnan(mm1))>0+
sum(np.abs(mm2)==np.inf)>0 + sum(np.isnan(mm2))>0)

if ann==0 and len(mm1)>1 and len(mm1)==len(mm2):
    if np.std(mm1)>0 and np.std(mm2)>0:
        coef=100*(scp.pearsonr(mm1,mm2)[0])
        if coef>0:
            anamef="fralf_.tmp"
            fo = open(anamef, "w")
            fo.write(str(args[0])+ ' %s'%(coef)+'\n')
            fo.close()

        KoefA[hh]=coef

        #mm1=mm1*np.std(mm2)/np.std(mm1)
        Koef[hh]=-np.std(mm1-mm2)
        if not astar0==np.inf:
            arr_bbx[hh]=astar0+np.cumsum(arr_bbbxxx)
        else:
            arr_bbx[hh]=arr_bbbxxx.copy()
        #hh=hh+1

# if hh==Nhh:

try:
    wwwww=0
    while not wwwww:
        try:
            arrrx=hkl.load("ralfrez.rlf2")
            wwwww=1
        except:
            tm.sleep(0.1)

    if len(arrrx)>=Nproc:
        return np.asarray(arr_bbx[hh],float)
    else:
        print(' %s'%(coef))
        if coef>0:
            zresu=arr_bbx[hh].copy()
            zresu[1:]=np.diff(zresu)
            zresu[0]=0
            arrrx.append(np.asarray(zresu,float))
            wwwww=0
            while not wwwww:
                try:
                    hkl.dump(arrrx,"ralfrez.rlf2")
                    wwwww=1
                except:
                    tm.sleep(0.6)

except:
    arrrx=[]

def decimat(adat_):
    if Lo:
        adat_ = np.log(adat_)
        k = 0
        adat__ = np.zeros(int(len(adat_) / aDecm), float)
        for i in range(int(len(adat_) / aDecm)):
            adat__[k] = np.median(adat_[i * aDecm:i * aDecm + aDecm])
            k = k + 1

    array=adat__[1:len(adat__)].copy()

    indices = np.arange(len(array))
    non_nan_mask = ~np.isnan(array)
    nan_mask = np.isnan(array)

    interpolated_values = np.interp(indices[nan_mask], indices[non_nan_mask], array[non_nan_mask])

    array[nan_mask] = interpolated_values

    if Lo:
        return np.exp(array)
    else:
        return (array)

def RALFICella(*args_):
    www=0
    while (not ww):
        try:
            with open("maxmindat.tmp", "rb") as file:
                bin_argxs=file.read()
                file.close()
            args=bin_argxs
            [Nf,anI,NNew,hhhx,aMM,aNN,Numproc]=np.asarray(struct.unpack('7i', args[0:28]),np.int32)
            # anamef="maxmin_.tmp"
            # fo = open(anamef, "w")
            # fo.write(' %s'%(Numproc)+'\n')
            # fo.close()
            binZDat=np.asarray(struct.unpack('%sf'%(anI*Nf), args[28:(4*anI*Nf+28)]),np.float32)
            ZDat=binZDat.reshape((anI,Nf))
            ss0=np.asarray(struct.unpack('%si'%(int((len(args)-(4*anI*Nf+28))/4)), args[(4*anI*Nf+28):]),np.int32)

            www=1
        except:
            tm.sleep(0.1)

    ww=1
    #ZData_ =np.mean(ZDat[:,0:Nf-NNew],axis=0)
    # aa=RandomQ(Nf)
    # ss0=np.concatenate((aa, aa, aa))
    while (ww):
        dd=ZDat.copy()
        for ii in range(len(dd)):
            dd[ii]=np.cumsum(dd[ii])
        mdd4=dd*0
        aa=RandomQ(Nf)
        ss4=np.concatenate((aa, aa, aa))
        liix=np.zeros((anI,Nf),int)
        mdd4_ =np.zeros(Nf,float)
        mdd4_[0:Nf-NNew]=1
        for i in range(anI):
            liix[i]=ss4[ss0[i+hhhx+ww:i+Nf+hhhx+ww]].copy()
            dd[i]=(dd[ii])[liix[i]].copy()
            mdd4[i]=mdd4_[liix[i]].copy()

        astart=np.inf
        dd=dd.reshape((anI*Nf))
        astart=dd[0]
        dd[1:]=np.diff(dd)
        dd[0]=0
        dd=dd.reshape((anI,Nf))
        D=np.std(dd)
        dd0=dd.copy()

```



```
P=np.zeros(3,float)
PP=1

seq0=(dd.reshape(len(dd)*len(dd[0])))[1:]*np.ceil(0.5*(1/(mdd4.reshape(len(dd)*len(dd[0]))==1)[0:len(dd)*len(dd[0])-1]+1/(mdd4.reshape(len(dd)*len(dd[0]))==1)[1:]))
seq0=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seq0)),float)
seq0=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seq0)),float)

dd_AA=dd*0
dd_BB=dd*0
dd_CC=dd*0
dd_Num=dd*0
dd_CCmx=dd*0-1e32
dd_CCmn=dd*0+1e32

for fff in range(3):
    aa=RandomQ(Nf)
    aa=aa-np.mean(aa)
    ss4_=np.concatenate((aa, aa, aa))
    DD=[]
    for hhhc in range(anI):
        vvv=(ss4_[hhhc:hhhc+Nf]*(1-mdd4[hhhc])).copy()
        DD_.append(vvv[::-1].copy())
    DD=np.asarray(DD,float)
    DD_=(DD_/np.std(DD_))*D*3

    #DD_=DD_*0
    KY=1
    for ii in range(aNN):
        for jj in range(aMM):
            dd1=dd[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)].copy()
            dd1=dd1-np.mean(dd1)
            mdd4_=mdd4[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)].copy()

            seqA0=(dd1.reshape(len(dd1)*len(dd1[0])))[1:]*np.ceil(0.5*(1/(mdd4_.reshape(len(dd1)*len(dd1[0]))==1)[0:len(dd1)*len(dd1[0])-1]+1/(mdd4_.reshape(len(dd1)*len(dd1[0]))==1)[1:]))
            seqA0_ = mdd4_.reshape(len(dd1)*len(dd1[0]))*0
            seqA0_[0]=1
            seqA0_[1:]=(abs(seqA0)==np.inf)+np.isnan(seqA0)
            seqA0_-=seqA0_.reshape((len(dd1),len(dd1[0])))
            seqA0=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqA0)),float)
            seqA0=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqA0)),float)

            seqA=(dd1.reshape(len(dd1)*len(dd1[0])))[1:]*np.ceil(0.5*np.fabs(1/(1*(mdd4_.reshape(len(dd1)*len(dd1[0]))==1)[0:len(dd1)*len(dd1[0])-1]-1*(mdd4_.reshape(len(dd1)*len(dd1[0]))==1)[1:]))
            seqA_ = mdd4_.reshape(len(dd1)*len(dd1[0]))*0
            seqA_[0]=1
            seqA_[1:]=(abs(seqA)==np.inf)+np.isnan(seqA)
            seqA_-=seqA_.reshape((len(dd1),len(dd1[0])))
            seqA=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqA)),float)
            seqA=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqA)),float)

            DD__A=DD_[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)].copy()
            DD__B=DD__A.copy()#
            # KY=KY*(-1)
            DD__A=KY*DD__A*(DD__A>0)
            DD__B=KY*DD__B*(DD__B>0)

            if len(dd1)>1 and len(dd1[0])>=len(dd1):
                eeB= XFilter( DD__A+dd1)-(DD__A) #,0)#,1)#
                eeA=-XFilter( DD__B-dd1)+(DD__B) #,0)#,1)#
                eeB=(eeB-eeA)/2
                # asr1=abs(eeB-xxxx)>abs(eeA-xxxx)
                # asr2=abs(eeB-xxxx)<abs(eeA-xxxx)
                # eeB=eeB*asr1+eeA*asr2+(eeB+eeA)*(asr1==asr2)/2

                # # eeB = faddeev_residual_refinement(eeB, eeA)
                # # eeB=eeB-eeA
                eeA=eeB.copy()

                dd_AA[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]=(eeA).copy()#*(eeB>0)*((eeA+eeB)>0)
                dd_BB[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]=(eeB).copy()#*(eeA<0)*((eeA+eeB)<0)
            dd2_1=(dd_AA[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)].copy()
            seqB=(dd2_1.reshape(len(dd2_1)*len(dd2_1[0])))[1:]*np.ceil(0.5*np.fabs(1/(1*(mdd4_.reshape(len(dd2_1)*len(dd2_1[0]))==1)[0:len(dd2_1)*len(dd2_1[0])-1]-1*(mdd4_.reshape(len(dd2_1)*len(dd2_1[0]))==1)[1:]))
            seqB=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqB)),float)
            seqB=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqB)),float)
            dd2_2=(dd_BB[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)].copy()
            seqC=(dd2_2.reshape(len(dd2_2)*len(dd2_2[0])))[1:]*np.ceil(0.5*np.fabs(1/(1*(mdd4_.reshape(len(dd2_2)*len(dd2_2[0]))==1)[0:len(dd2_2)*len(dd2_2[0])-1]-1*(mdd4_.reshape(len(dd2_2)*len(dd2_2[0]))==1)[1:]))
            seqC=np.asarray(list(filter(lambda x: abs(x)!= np.inf, seqC)),float)
            seqC=np.asarray(list(filter(lambda x: abs(np.isnan(x))!= 1, seqC)),float)

try:
    P_1=P.copy()
    P_2=P.copy()
    P_1[0:2]=np.polyfit(seqA,seqB,1)
    P_2[0:2]=np.polyfit(seqA,seqC,1)

    # print(100*scp.pearsonr(seqA,seqB)[0])
    P_1[0]=np.std(seqB)/np.std(seqA)
    P_1[1]=np.mean(seqB)-P_1[0]*np.mean(seqA)
    P_1[2]=0
    P_2[0]=np.std(seqC)/np.std(seqA)
    P_2[1]=np.mean(seqC)-P_2[0]*np.mean(seqA)
    P_2[2]=0
    if P_1[0]>0 and P_2[0]>0:# and 100*scp.pearsonr(seqA,seqB)[0]>0 and 100*scp.pearsonr(seqA,seqC)[0]>0:
        aanum=dd_Num[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)].copy()
        seqAA=dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]+np.maximum(dd_CCmx[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]-dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)],(dd2_1.copy()-P_1[1])/P_1[0]-dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)])
        seqBB=dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]+np.minimum(dd_CCmn[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]-dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)],(dd2_2.copy()-P_2[1])/P_2[0]-dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)])
        dd_CCmx[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]=np.maximum((dd_CCmx[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]),seqAA)
        dd_CCmn[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]=np.minimum((dd_CCmn[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]),seqBB)
        dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]=((dd0[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]+aanum+(dd_CCmx[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]+dd_CCmn[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]))/2)/(aanum+1)
        dd_Num[int(ii*anI/aNN):int((ii+1)*anI/aNN),int(jj*Nf/aMM):int((jj+1)*Nf/aMM)]=aanum+1
except:
    PP=0
# dd_CCmx=filter_ralf1_matrix(dd_CCmx, intensity=1.0, iterations=100)
# dd_CCmn=filter_ralf1_matrix(dd_CCmn, intensity=1.0, iterations=100)
if not PP==0:
    dd0=(dd_CCmx+dd_CCmn)/2
    asr1=abs(dd_CCmx-dd0)>abs(dd_CCmn-dd0)
    asr2=abs(dd_CCmx-dd0)<abs(dd_CCmn-dd0)
    dd_CCmx=dd_CCmx*asr1+dd_CCmn*asr2+(dd_CCmx+dd_CCmn)*(asr1==asr2)/2
    dd_CCmn=dd_CCmx.copy()
    if not astart==np.inf:
        # dd_AA=dd_CC*PP*(dd_CC>0)
        # dd_BB=dd_CC*PP*(dd_CC<0)
        dd_AA=dd_CCmx.reshape((anI*Nf))
        dd_AA=astart+np.cumsum(dd_AA)
        dd_AA=dd_AA.reshape((anI,Nf))
        dd_BB=dd_CCmn.reshape((anI*Nf))
        dd_BB=astart+np.cumsum(dd_BB)
        dd_BB=dd_BB.reshape((anI,Nf))

    dd_AA=(dd_AA+dd_BB)
    dd_BB=dd_AA.copy()
    aMx=np.zeros(Nf,float)-1e32
    aMn=np.zeros(Nf,float)+1e32
    for i in range(anI):
        aMx[liix[i]]=np.maximum(aMx[liix[i]],dd_AA[i])
        aMn[liix[i]]=np.minimum(aMn[liix[i]],dd_BB[i])

    aRes=np.asarray([aMx, aMn],np.float32)

    aRes[0][1:]=np.diff(aRes[0])
    aRes[0][0]=0
```



```

        aRes[1][1:]=np.diff(aRes[1])
        aRes[1][0]=0
        ww=0

    www=0
    while (not www):
        try:
            with open("maxmin.tmp", "rb") as file:
                areza=file.read()
                file.close()
                www=1

        except:
            tm.sleep(0.1)

    nsz=int(len(areza)/2/4/Nf)
    if nsz<Numproc:
        www=0
        while (not www):
            try:
                areza=np.asarray(struct.unpack('%sf'%(int(len(areza)/4)), areza),np.float32)
                areza=np.asarray(areza.reshape((nsz,2,Nf)),np.float32)
                areza=np.zeros((nsz+1,2,Nf),np.float32)
                areza_[0:nsz]=areza.copy()
                areza_[nsz]=aRes.copy()

                with open("maxmin.tmp", "wb") as file:
                    ssssss=np.asarray(areza_.reshape(((nsz+1)*2*Nf)),np.float32)
                    sssss=struct.pack("%sf"%(len(ssssss)),*ssssss)
                    file.write(sssss)
                    file.close()

                www=1

            except:
                tm.sleep(0.1)

        ww=ww+1
    else:
        ww=0
        www=0
        while (not www):
            try:
                # anamef="maxmin.x.tmp"
                # fo = open(anamef, "w")
                # fo.write('%s '%(aRes)+'\n')
                # fo.close()
                www=1

            except:
                tm.sleep(0.1)

        break
    #os._exit(os.EX_OK)
    return aRes

def RALF1Calculation(ar0,arr_z,Lengt,Nf,NNew,nn):
    aTmStop=1
    NIt=3
    lSrez=0.99
    dNIt=4
    KPP=0
    #key=13
    Arr_AAA=np.zeros((Niter*Nproc,Nf),float)
    arr_rezBz=np.zeros(Nf,float)
    arr_rezBzz=arr_rezBz.copy()

    arr_z[Nf-NNew:]=arr_z[Nf-NNew-1]
    all_RezN=np.zeros((Ngroup,Niter,Nf),float)+np.Inf
    all_RezM=np.zeros((Ngroup,Niter,Nf),float)-np.Inf
    dd1a=np.zeros((Ngroup,Niter,Nf),float)-np.Inf
    dd2a=np.zeros((Ngroup,Niter,Nf),float)+np.Inf
    all_RezNM=np.zeros((Ngroup,Niter,Nf),float)
    all_RezMM=np.zeros((Ngroup,Niter,Nf),float)
    #args=[0] for j in range(Nproc)]

    hh0=0
    hhh=0
    hhh_=0

    Koef_=[]
    ZZ=0
    key=0
    while hhh<aTmStop and not key == 13:
        if hhh==int(Niter/3):
            if hhh<aTmStop-1:
                try:
                    os.remove(wrkdir + "rez.rlf1")
                except:
                    hhh=hhh_
                    nnn=int(nn*0.5)
                    aTmStop=6
                    hh0=0
                    hhh=0
                    arr_z=np.zeros(Nf+nnn,float)
                    arr_z[0:nnn]=arr_rezBz[0:nnn].copy()
                    arr_z[nnn:Nf]=arr_rezBz[nnn:Nf].copy()
                    arr_z[Nf:]=arr_z[Nf-1]
                    Lengt=Lengt+nnn
                    ar0=arr_z[0:Lengt-nnn].copy()
                    Nf=Nf+nnn
                    arr_rezBz=np.zeros(Nf,float)
                    arr_rezBzz=arr_rezBz.copy()
                    Arr_AAA=np.zeros((Niter*Nproc,Nf),float)
                    arr_z[Nf-NNew:]=arr_z[Nf-NNew-1]
                    all_RezN=np.zeros((Ngroup,Niter,Nf),float)+1e32
                    all_RezM=np.zeros((Ngroup,Niter,Nf),float)-1e32
                    dd1a=np.zeros((Ngroup,Niter,Nf),float)-1e32
                    dd2a=np.zeros((Ngroup,Niter,Nf),float)+1e32
                    all_RezNM=np.zeros((Ngroup,Niter,Nf),float)
                    all_RezMM=np.zeros((Ngroup,Niter,Nf),float)
                    hhh=hhh_+1

            else:
                hhh=hhh_+1
                ZZ=1

    if ZZ==0:
        try:
            [hhha,Arr_AAA]=(hkl.load(wrkdir + "rez.rlf1"))
        except:
            hhha=hh0-1

    if hh0>=hhha:
        if Lo:
            arr_A=np.log(arr_z)
        else:
            arr_A=arr_z.copy()

        Asr=np.mean(arr_A[0:Nf-NNew])
        arr_A=arr_A-Asr
        Klq=np.power(10,np.floor(np.log10(np.max(abs(arr_A)))))
        arr_A=arr_A/Klq

        NChan=1
        #####
        ssssss=np.asarray([NChan, Nf, NNew, NIt, Nproc],np.int32)
        sssss=np.asarray(arr_A,np.float32)

```



```

argss=(struct.pack("%s!"%(len(ssssss)),*ssssss)+
        struct.pack("%sf"%(Nf),*sssss))
with open("filtrx.tmp", "wb") as file:
    file.write(argss)
    file.close()

try:
    wwww=1
    while wwww and wwww<100:
        try:
            arezAMx=hkl.load("ralfrez.rlf2")
            wwww=0
        except:
            tm.sleep(0.1)
            wwww=wwww+1

    if wwww>0:
        arezAMx_=[]

    if len(arezAMx_)==Nproc or arezAMx_==[]:
        wwww=1
        while wwww and wwww<100:
            try:
                hkl.dump([], "ralfrez.rlf2")
                wwww=0
            except:
                tm.sleep(0.6)
                wwww=wwww+1

except:
    wwww=0
    while not wwww:
        try:
            hkl.dump([], "ralfrez.rlf2")
            wwww=1
        except:
            tm.sleep(0.6)

arezAMx_=[]
# for iProc in range(Nproc):
#     aaa=RALf1FiltrQ(zip(argss))
#     arezAMx_.append(aaa)

pool = mp.Pool(processes=Nproc)
try:
    #pool.map(RALf1FiltrQ, argss)
    pool.map_async(RALf1FiltrQ, zip(argss))
    wwwww=0
    while not wwwww:
        wwwww=0
        while not wwwww:
            try:
                arezAMx=hkl.load("ralfrez.rlf2")
                wwwww=1
            except:
                tm.sleep(0.1)
        nsz=len(arezAMx_)
        if nsz>=Nproc:
            wwwww=1
            pool.terminate()
        else:
            tm.sleep(0.1)

except:
    arezAMx=hkl.load("ralfrez.rlf2")
#arezAMx= np.asarray(arezAMx,float)[0,:,:)
del(pool)

if arezAMx_==[]:
    wwwww=0
    while not wwwww:
        try:
            arezAMx=hkl.load("ralfrez.rlf2")
            wwwww=1
        except:
            tm.sleep(0.1)

if len(arezAMx_)>0:
    wwwww=0
    while not wwwww:
        try:
            hkl.dump(arezAMx_, "ralfrez_.rlf2")
            wwwww=1
        except:
            tm.sleep(0.6)

arezAMx= np.asarray(arezAMx_,float)

wwwww=0
while not wwwww:
    try:
        hkl.dump([], "ralfrez.rlf2")
        wwwww=1
    except:
        tm.sleep(0.6)

arezAMx= np.asarray(arezAMx,float)*Klg+Asr

for iGr in range(Ngroup):
    Arr_AAA[iGr*NIter*int(Nproc/Ngroup)+hh0*int(Nproc/Ngroup):iGr*NIter*int(Nproc/Ngroup)+(hh0+1)*int(Nproc/Ngroup)]=(
        arezAMx[int(iGr*(Nproc/Ngroup)):int((iGr+1)*(Nproc/Ngroup))]).copy()

hkl.dump([hh0+1,Arr_AAA], wrkdir + "rez.rlf1")
[hhha,Arr_AAA]=(hkl.load(wrkdir + "rez.rlf1"))

WrtTodr=1
if hhh>=hhha-1:
    #WrtTodr=1
    aDur=4

aNN=3
aMM=3

for iGr in range(Ngroup):
    ZDat=Arr_AAA[iGr*NIter*int(Nproc/Ngroup)+max(0,0)*int(Nproc/Ngroup):iGr*NIter*int(Nproc/Ngroup)+(hhha))*int(Nproc/Ngroup)].copy()
    if iGr==0:
        xxxx=ZDat.copy()
    else:
        xxxx=np.concatenate((xxxx, ZDat))
ZDat=xxxx.copy()
anI=len(ZDat)
# for i in range(anI):
#     ZDat[i]= savgol_filter(ZDat[i], 65, 5)
for ii in range(3):
    ar0x=np.median(ZDat,axis=0)
    ar0x_=0.4*(np.median(abs((ZDat)-(ar0x)),axis=0))

lnn=len(ZDat[0])
NNew=int(.35*lnn)
for i in range(anI):
    for j in range(lnn):
        if not abs(ZDat[i,j]-(ar0x[j]))<=ar0x_[j]:
            if ZDat[i,j]<((ar0x[j])-ar0x_[j]):
                ZDat[i,j]=(ar0x[j])-ar0x_[j]

```



```

        else:
            if ZDat[i,j]>((ar0x[j])+ar0x_[j]):
                ZDat[i,j]=(ar0x[j])+ar0x_[j]

P=np.zeros(3,float)
# for i in range(anI):
#     dd=ZDat[i][lnn-NNew:].copy()
#     x=ar0x[lnn-NNew:lnn-NNew+int(lSrez*(NNew-(lnn-len(ar0x))))].copy()
#     ZDat[i][lnn-NNew:]=filterFourierQ(ZDat[i],(ar0x),NNew,1)[lnn-NNew:]
#     # ZDat[i]= savgol_filter(ZDat[i], 65, 5)
#     P[0:2]=np.polyfit(x,ZDat[i][lnn-NNew:lnn-NNew+int(lSrez*(NNew-(lnn-len(ar0x))))],1)
#     if not P[0]>0:
#         P[0:2]=np.polyfit(dd,ZDat[i][lnn-NNew:],1)
#     ZDat[i][lnn-NNew:]=(ZDat[i][lnn-NNew:]-P[1])/P[0]
#     ZDat[i]= savgol_filter(ZDat[i], 14, 5)

if Lo:
    ar0x=np.exp(ar0x)
ar0x[0:len(ar0)]=ar0[0:len(ar0)].copy()

if Lo:
    dd=np.log(ar0x)
else:
    dd=ar0x.copy()

astart0=np.inf

if Lo:
    ar0_=np.exp(dd_)
else:
    ar0_=dd_.copy()
MMM_=0
tm0=tm.time()
tm1=0
aMx0=dd_*0-np.inf
aMn0=dd_*0+np.inf
mml=dd_*0
mm2=mm1.copy()
while MMM<2*Nproc and (tm1-tm0)<MxTime:
    arr_RezM= np.zeros((Ngroup,Nf),float)
    arr_RezN= np.zeros((Ngroup,Nf),float)
    MMM=0
    for iGr in range(Ngroup):
        ZDat=Arr_AAA[iGr*NIter*int(Nproc/Ngroup)+max(0,0)*int(Nproc/Ngroup):(iGr*NIter*int(Nproc/Ngroup)+(hhha))*int(Nproc/Ngroup)].copy()
        if Lo:
            ZDat=np.exp(ZDat)
            xxxx=ZDat.copy()
            for i in range(aNN-1):
                xxxx=np.concatenate((xxxx, ZDat))
            ZDat=xxxx.copy()
            hhhx=0
            anI=len(ZDat)
            # for i in range(anI):
            #     ZDat[i][:len(ar0)]=ar0.copy()
            #     if Lo:
            #         ZDat[i]= np.exp(savgol_filter(np.log(ZDat[i]), 34, 5))
            #     else:
            #         ZDat[i]= savgol_filter(ZDat[i], 34, 5)
            #     lnn=len(ZDat[0])
            for i in range(anI):
                for j in range(lnn):
                    if Lo:
                        ZDat[i]=np.log(ZDat[i])
                        if not abs(ZDat[i,j]-np.log(ar0x[j]))<=ar0x_[j]:
                            if ZDat[i,j]<(np.log(ar0x[j])-ar0x_[j]):
                                ZDat[i,j]=np.log(ar0x[j])-ar0x_[j]
                            else:
                                if ZDat[i,j]>(np.log(ar0x[j])+ar0x_[j]):
                                    ZDat[i,j]=np.log(ar0x[j])+ar0x_[j]
                                ZDat[i]=np.exp(ZDat[i])
                    else:
                        if not abs(ZDat[i,j]-(ar0x[j]))<=ar0x_[j]:
                            if ZDat[i,j]<((ar0x[j])-ar0x_[j]):
                                ZDat[i,j]=(ar0x[j])-ar0x_[j]
                            else:
                                if ZDat[i,j]>((ar0x[j])+ar0x_[j]):
                                    ZDat[i,j]=(ar0x[j])+ar0x_[j]

if hhh==0:
    if Lo:
        arr_rezBzz=np.exp(np.median(np.log(ZDat),axis=0))
    else:
        arr_rezBzz=np.median(ZDat,axis=0)
for i in range(anI):
    if not astart0==np.inf:
        if Lo:
            ZDat[i][1:]=np.diff(np.log(ZDat[i])+KPP*np.log(arr_rezBzz))/(1+KPP)
        else:
            ZDat[i][1:]=np.diff(ZDat[i]+KPP*arr_rezBzz)/(1+KPP)
            ZDat[i][0]=0
    else:
        if Lo:
            ZDat[i]=(np.log(ZDat[i])+KPP*np.log(arr_rezBzz))/(1+KPP)
        else:
            ZDat[i]=(ZDat[i]+KPP*arr_rezBzz)/(1+KPP)
P=np.zeros(3,float)
for i in range(anI):
    dd=ZDat[i][Nf-NNew:].copy()
    if Lo:
        x=np.log(ar0_[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))])
        ZDat[i][Nf-NNew:]=filterFourierQ(ZDat[i],np.log(ar0_),NNew,1,-1,1)[Nf-NNew:]
    else:
        x=ar0_[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))].copy()
        ZDat[i][Nf-NNew:]=filterFourierQ(ZDat[i],(ar0_),NNew,1,-1,1)[Nf-NNew:]
    P[0:2]=np.polyfit(x,ZDat[i][Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))],1)
    if not P[0]>0:
        # P[0:2]=np.polyfit(dd,ZDat[i][Nf-NNew:],1)
        P[0]=np.std(ZDat[i][Nf-NNew:])/np.std(dd)
        P[1]=np.mean(ZDat[i][Nf-NNew:])-P[0]*np.mean(dd)
    else:
        P[0]=np.std(ZDat[i][Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))])/np.std(x)
        P[1]=np.mean(ZDat[i][Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))]-P[0])*np.mean(x)

    ZDat[i][Nf-NNew:]=(ZDat[i][Nf-NNew:]-P[1])/P[0]

if anI<aNN:
    all_RezM[iGr][hhh]=np.amax(ZDat,axis=0)
    all_RezN[iGr][hhh]=np.amin(ZDat,axis=0)
else:
    aMx_=0
    aMn_=0
    mdd4=ZDat*0
    aa=RandomQ(Nf)
    ss0=np.concatenate((aa, aa, aa))
    hhhx=0
    #pool = mp.Pool(processes=Numproc)
    while hhhx<int(NIter/12):
        #if dNIit*int(hhhx/dNIit)==hhhx:
        areza=[]

        ssssss=np.asarray([Nf,anI,NNew,hhhx,aMM,aNN,Numproc],np.int32)
        aSred=np.mean(ZDat)
        sssss=np.asarray(ZDat.reshape((anI*Nf))-aSred,np.float32)

```



```

ssss=np.asarray(ss0,np.int32)

arrgsx=(struct.pack("%s!"%(len(ssssss)),*ssssss)+
        struct.pack("%sf"%(anI*Nf),*sssss)+
        struct.pack("%s!"%(len(ss0)),*ssss))
with open("maxmindat.tmp", "wb") as file:
    file.write(arrgsx)
    file.close()
# RALF1Cella(zip(arrgsx))

wwwww=0
while not wwwww:
    with open("maxmin.tmp", "wb") as file:
        file.close()
    pool = mp.Pool(processes=Numproc)
    resa=pool.map_async(RALF1Cella,zip(arrgsx))
    wwwww=0
    while not wwwww:
        with open("maxmin.tmp", "rb") as file:
            areza=file.read()
            file.close()
            areza=np.asarray(struct.unpack('%sf'%(int(len(areza)/4)), areza),np.float32)

        try:
            areza=np.asarray(areza.reshape((int(len(areza)/2/Nf),2,Nf)),np.float32)+aSred
            nsz=len(areza)

            if nsz>=Numproc:
                wwwww=1
                wwwwww=1
                pool.terminate()
            else:
                tm.sleep(0.1)
        except:
            wwwww=1
            pool.terminate()

#del(pool)

areza=np.asarray(areza,float)
PP=1
aMx=np.amax(areza[:,0],axis=0)
aMn=np.amin(areza[:,1],axis=0)

aMx=savgol_filter(aMx, 14, 5)
aMn=savgol_filter(aMn, 14, 5)

if not PP==0:
    if dNIt*int(hhhx/dNIIt)==hhhx:
        aMx_=aMx.copy()
        aMn_=aMn.copy()
        aMx0=aMx_.copy()
        aMn0=aMn_.copy()

        # aMx_=np.maximum(aMx_,aMx)
        # aMn_=np.minimum(aMn_,aMn)
        aMx_=(aMx_*(hhhx-dNIIt*int(hhhx/dNIIt))+np.maxium(aMx_,aMx))/(hhhx-dNIIt*int(hhhx/dNIIt)+1)
        aMn_=(aMn_*(hhhx-dNIIt*int(hhhx/dNIIt))+np.minimum(aMn_,aMn))/(hhhx-dNIIt*int(hhhx/dNIIt)+1)

        if Lo:
            x=np.log(ar0_[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))])
            dd1=filterFourierQ(aMx_,np.log(ar0_),NNew,1)
            dd2=filterFourierQ(aMn_,np.log(ar0_),NNew,1)

        else:
            x=ar0_[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))].copy()
            dd1=filterFourierQ(aMx_,(ar0_),NNew,1)
            dd2=filterFourierQ(aMn_,(ar0_),NNew,1)

        dd0=(dd1+dd2)/2
        asr1=abs(dd1-dd0)>abs(dd2-dd0)
        asr2=abs(dd1-dd0)<abs(dd2-dd0)
        dd1=dd1*asr1+dd2*asr2+(dd1+dd2)*(asr1=asr2)/2
        dd2=dd1.copy()

        if Lo:
            y_1=dd1[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))].copy()
            y_2=dd2[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))].copy()

        else:
            y_1=dd1[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))].copy()
            y_2=dd2[Nf-NNew:Nf-NNew+int(lSrez*(NNew-(Nf-len(ar0))))].copy()

        P_1=P.copy()
        P_2=P.copy()
        try:
            P_1[0:2]=np.polyfit(x,y_1,1)
            P_2[0:2]=np.polyfit(x,y_2,1)

            if not P_1[0]>0 or not P_2[0]>0 or not abs(P_1[0]-P_2[0])/(abs(P_1[0])+abs(P_2[0]))<0.2 or 100*scp.pearsonr(x,y_1)[0]<10 or 100*scp.pearsonr(x,y_2)[0]<10:
                PP=0
            # else:
            #     P_1[0]=np.std(y_1)/np.std(x)
            #     P_1[1]=np.mean(y_1)-P_1[0]*np.mean(x)
            #     P_2[0]=np.std(y_2)/np.std(x)
            #     P_2[1]=np.mean(y_2)-P_2[0]*np.mean(x)

        except:
            PP=0
if not PP==0:
    arr_RezM[iGr][Nf-NNew:]=(dd1[Nf-NNew:]-P_1[1])/P_1[0]
    arr_RezN[iGr][Nf-NNew:]=(dd2[Nf-NNew:]-P_2[1])/P_2[0]
    if Lo:
        arr_RezM[iGr][:Nf-NNew]=np.log(ar0_[[:Nf-NNew]])
        arr_RezN[iGr][:Nf-NNew]=np.log(ar0_[[:Nf-NNew]])
    else:
        arr_RezM[iGr][:Nf-NNew]=ar0_[[:Nf-NNew]].copy()
        arr_RezN[iGr][:Nf-NNew]=ar0_[[:Nf-NNew]].copy()
    aMx0=aMx_.copy()
    aMn0=aMn_.copy()

    if not PP==0:
        if dNIIt*int(hhhx/dNIIt)==hhhx:
            all_RezM[iGr][hhh]=arr_RezM[iGr].copy()
            all_RezN[iGr][hhh]=arr_RezN[iGr].copy()
        else:
            all_RezM[iGr][hhh]=np.maximum(all_RezM[iGr][hhh],arr_RezM[iGr])
            all_RezN[iGr][hhh]=np.minimum(all_RezN[iGr][hhh],arr_RezN[iGr])
        if hhhx==0:
            dd1a[iGr,hhh]=(all_RezM[iGr][hhh]).copy()#-P[1])/P[0]
            dd2a[iGr,hhh]=(all_RezN[iGr][hhh]).copy()#-P[1])/P[0]
        else:
            dd1a[iGr,hhh]=(dd1a[iGr,hhh]*hhhx+(all_RezM[iGr][hhh]))/(hhhx+1)#-P[1])/P[0]/(hhhx+1)
            dd2a[iGr,hhh]=(dd2a[iGr,hhh]*hhhx+(all_RezN[iGr][hhh]))/(hhhx+1)#-P[1])/P[0]/(hhhx+1)
            hhhx=hhhx+1
        else:
            PP=0
if PP==0:
    aMx_=aMx0.copy()
    aMn_=aMn0.copy()
    aa=RandomQ(Nf)
    ss0=np.concatenate((aa, aa, aa))

tm1=tm.time()
if (tm1-tm0)>MxTime:

```



```

        break
    del(pool)
tm1=tm.time()
if (tm1-tm0)>MxTime:
    break

dd1=np.amax(dd1a[iGr,max(0,(hhh+1)-int(dNIt/2+1)):hhh+1],axis=0)
dd2=np.amin(dd2a[iGr,max(0,(hhh+1)-int(dNIt/2+1)):hhh+1],axis=0)

if Lo:
    arr_RezM[iGr][Nf-NNew:]=dd1[Nf-NNew:]
    arr_RezN[iGr][Nf-NNew:]=dd2[Nf-NNew:]
else:
    arr_RezM[iGr][Nf-NNew:]=dd1[Nf-NNew:]
    arr_RezN[iGr][Nf-NNew:]=dd2[Nf-NNew:]

if Lo:
    x=np.log(ar0_[Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))])
else:
    x=ar0_[Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))].copy()

y_1=arr_RezM[iGr][Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))].copy()
y_2=arr_RezN[iGr][Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))].copy()
P_1[0:2]=np.polyfit(x,y_1,1)
P_2[0:2]=np.polyfit(x,y_2,1)

PP=(abs(P_1[0]-1)>1) or (abs(P_2[0]-1)>1)
P_1[0]=np.std(y_1)/np.std(x)
P_1[1]=np.mean(y_1)-P_1[0]*np.mean(x)
P_2[0]=np.std(y_2)/np.std(x)
P_2[1]=np.mean(y_2)-P_2[0]*np.mean(x)

all_RezNM[iGr][hhh][Nf-NNew:]=0.5*((arr_RezM[iGr][Nf-NNew:]-P_1[1])/P_1[0]
+(arr_RezN[iGr][Nf-NNew:]-P_2[1])/P_2[0])

if not astart0==np.inf:
    all_RezMM[iGr][hhh]=np.cumsum(all_RezNM[iGr][hhh])
else:
    all_RezMM[iGr][hhh]=all_RezNM[iGr][hhh].copy()

if Lo:
    all_RezMM[iGr][hhh][Nf-NNew:]=(filterFourierQ((all_RezMM[iGr][hhh]),np.log(ar0_),NNew,1))[Nf-NNew:]
else:
    all_RezMM[iGr][hhh][Nf-NNew:]=(filterFourierQ((all_RezMM[iGr][hhh]),(ar0_),NNew,1))[Nf-NNew:]
if Lo:
    all_RezMM[iGr][hhh][Nf-NNew:]=all_RezMM[iGr][hhh][Nf-NNew:]
else:
    all_RezMM[iGr][hhh][Nf-NNew:]=all_RezMM[iGr][hhh][Nf-NNew:]

if Lo:
    x=np.log(ar0[Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))])
else:
    x=ar0[Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))].copy()

y=all_RezMM[iGr][hhh][Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))].copy()

P[0:2]=np.polyfit(x,y,1)
#abs(P[0]-1)>1 or

if abs(P[0]-1)>1 or 100*scp.pearsonr(x,all_RezMM[iGr][hhh][Nf-NNew:Nf-NNew+int(1Srez*(NNew-(Nf-len(ar0))))])[0]<20:
    MMM=MMM+1
all_RezMM[iGr][hhh][Nf-NNew:]=(all_RezMM[iGr][hhh][Nf-NNew:]-P[1])/P[0]

if Lo:
    all_RezMM[iGr][hhh][:Nf-NNew]=np.log(ar0[:Nf-NNew])
else:
    all_RezMM[iGr][hhh][:Nf-NNew]=ar0[:Nf-NNew].copy()

arr_RezM[iGr]=(np.amax(all_RezMM[iGr][0:hhh+1],axis=0)+np.amin(all_RezMM[iGr][0:hhh+1],axis=0))/2
arr_RezM[iGr]= savgol_filter(arr_RezM[iGr], 14, 5)

tm1=tm.time()
if (tm1-tm0)>MxTime:
    break
MMM=int(2*MMM/Ngroup)
arr_rezBz=np.mean(arr_RezM, axis=0)

if Lo:
    arr_rezBz=np.exp(arr_rezBz)
    for iGr in range(Ngroup):
        arr_RezM[iGr]=np.exp((arr_RezM[iGr]))

mm1=ar0[Nf-NNew:].copy()
mm2=arr_rezBz[Nf-NNew:len(ar0)].copy()
try:
    if 100*scp.pearsonr(mm1,mm2)[0]>10 and MMM==0:
        break
    else:
        MMM=MMM+1
except:
    break

tm1=tm.time()
if MMM<2*Nproc and np.std(mm1)>0 and np.std(mm2)>0 and (tm1-tm0)<MxTime:
    arr_rezBzz=arr_rezBz.copy()
    Kcoef.append(100*scp.pearsonr(mm1,mm2)[0])
    hhh=hhh+1

if hh0==2*NIter:
    hhh=NIter
    print (hhh+10000*hh0)
return arr_rezBz

# Data Fetching
def fetch_data(client, ticker=SYMBOL, limit=1000):
    try:
        df = yf.download(ticker, period=f"{limit}m", interval="1m")
        if df.empty:
            raise ValueError("No data fetched from yfinance")
        df = df[['Close']].rename(columns={'Close': 'close'})
        return df
    except Exception as e:
        logging.error(f"Data fetch error: {e}")
        return pd.DataFrame()

# BigQuery Operations with Retry Logic
def upload_to_bigquery(bq_client, df):
    try:
        # Prepare DataFrame for BigQuery
        df_bq = df.reset_index().rename(columns={'index': 'timestamp'})
        df_bq['timestamp'] = df_bq['timestamp'].astype(str) # Ensure timestamp is string for BigQuery

        # Define schema explicitly
        schema = [
            bigquery.SchemaField("timestamp", "TIMESTAMP"),
            bigquery.SchemaField("close", "FLOAT"),
        ]

        # Configure table and load job
        table_id = f"{BQ_PROJECT}.{BQ_DATASET}.{BQ_TABLE}"
        job_config = bigquery.LoadJobConfig(
            schema=schema,

```



```

        write_disposition="WRITE_APPEND",
        time_partitioning=bigquery.TimePartitioning(type_=bigquery.TimePartitioningType.DAY),
    )

    # Load data to BigQuery
    job = bq_client.load_table_from_dataframe(df_bq, table_id, job_config=job_config)
    job.result() # Wait for job to complete
    logging.info(f"Uploaded {len(df_bq)} rows to {table_id}")
except Exception as e:
    logging.error(f"BigQuery upload error: {e}")
    raise

def download_from_bigquery(bq_client):
    try:
        table_id = f"{BQ_PROJECT}.{BQ_DATASET}.{BQ_TABLE}"
        query = f"""
            SELECT timestamp, close
            FROM `{table_id}`
            WHERE timestamp IS NOT NULL
            ORDER BY timestamp
        """
        df = bq_client.query(query).to_dataframe()
        if df.empty:
            logging.warning("No data retrieved from BigQuery")
            return pd.DataFrame()

        df['timestamp'] = pd.to_datetime(df['timestamp'])
        df.set_index('timestamp', inplace=True)
        df = df[['close']] # Ensure only 'close' column is returned
        return df
    except Exception as e:
        logging.error(f"BigQuery download error: {e}")
        return pd.DataFrame()

# LSTM Model Setup
def create_lstm_model(window_size):
    model = Sequential([
        LSTM(50, input_shape=(window_size, 1), return_sequences=False),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse')
    return model

# Signal Generation and Trading
def get_real_time_price(client):
    try:
        return float(client.get_symbol_ticker(symbol="BTCUSD")['price'])
    except Exception as e:
        logging.error(f"Price fetch error: {e}")
        return None

def generate_signal(current_price, predicted_price, trailing_price=None):
    if trailing_price is None:
        trailing_price = current_price
    if predicted_price > current_price * (1 + TAKE_PROFIT_PCT):
        return 'buy', current_price
    elif predicted_price < trailing_price * (1 - TRAILING_STOP_PCT):
        return 'sell', current_price
    return 'hold', trailing_price

def execute_trade(client, signal, quantity=0.001):
    try:
        if signal == 'buy':
            order = client.order_market_buy(symbol='BTCUSD', quantity=quantity)
            logging.info(f"Buy order executed: {order}")
            return order
        elif signal == 'sell':
            order = client.order_market_sell(symbol='BTCUSD', quantity=quantity)
            logging.info(f"Sell order executed: {order}")
            return order
        return None
    except Exception as e:
        logging.error(f"Trade error: {e}")
        return None

# Notifications
def send_email(subject, body):
    try:
        msg = MIMEText(body)
        msg['Subject'] = subject
        msg['From'] = EMAIL_SENDER
        msg['To'] = EMAIL_RECEIVER
        with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
            server.starttls()
            server.login(EMAIL_SENDER, EMAIL_PASSWORD)
            server.sendmail(EMAIL_SENDER, EMAIL_RECEIVER, msg.as_string())
        logging.info("Email sent successfully")
    except Exception as e:
        logging.error(f"Email error: {e}")

def send_telegram(message):
    try:
        url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
        requests.post(url, data={"chat_id": TELEGRAM_CHAT_ID, "text": message})
        logging.info("Telegram message sent")
    except Exception as e:
        logging.error(f"Telegram error: {e}")

def arima_plus_forecast(data):
    try:
        model = ARIMA(data, order=(5, 1, 0))
        model_fit = model.fit()
        forecast = model_fit.forecast(steps=1)
        return forecast[0]
    except Exception as e:
        logging.error(f"ARIMA forecast error: {e}")
        return data[-1] # Fallback to last known value

def ralfi_filter_forecast(data):
    # Argument setup using clarified/corrected approach:
    Lengt = len(data)
    Nf = int(Lengt * 1.2) if Lengt > 0 else 1
    # Nf must be >= Lengt
    NNew = int(Lengt * 0.5)
    nn = int(Nf * 0.3)

    arr_z = np.zeros(Nf)
    arr_z[:Lengt] = data
    arr_z[Lengt:] = arr_z[Lengt-1] if Lengt > 0 else 0

    # The function expects:
    # ar0, arr_z, Lengt, Nf, NNew, nn, aTmStop, gray_sz1, gray_sz2
    return RALFICalculation(
        ar0=data,
        arr_z=arr_z,
        Lengt=Lengt,
        Nf=Nf,
        NNew=NNew,
        nn=nn
    )

# Trading Bot GUI
class TradingBotGUI:

```



```

def __init__(self, master, client, bq_client, model, window_size):
    self.master = master
    self.client = client
    self.bq_client = bq_client
    self.model = model
    self.window_size = window_size
    self.data = fetch_data(client)
    self.running = False
    self.trailing_price = None

    # GUI Elements
    self.use_ralfl = tk.BooleanVar(value=False)
    tk.Checkbutton(master, text="Use RALF1 Filter", variable=self.use_ralfl).pack()
    tk.Label(master, text="Current Price:").pack()
    self.price_label = tk.Label(master, text="--")
    self.price_label.pack()
    tk.Label(master, text="Predicted Price:").pack()
    self.pred_label = tk.Label(master, text="--")
    self.pred_label.pack()
    tk.Button(master, text="Upload to BigQuery", command=self.upload_data).pack()
    tk.Button(master, text="Start Bot", command=self.start_bot).pack()
    tk.Button(master, text="Stop Bot", command=self.stop_bot).pack()

    # Plot
    self.fig, self.ax = plt.subplots(figsize=(8, 4))
    self.canvas = FigureCanvasTkAgg(self.fig, master=master)
    self.canvas.get_tk_widget().pack()

def upload_data(self):
    if not self.data.empty():
        upload_to_bigquery(self.bq_client, self.data)

def start_bot(self):
    self.running = True
    threading.Thread(target=self.run_bot, daemon=True).start()

def stop_bot(self):
    self.running = False

def run_bot(self):
    while self.running:
        if not (START_HOUR <= time.gmtime().tm_hour <= END_HOUR):
            logging.info("Outside trading window")
            time.sleep(60)
            continue

        # Fetch latest data
        self.data = fetch_data(self.client)
        if self.data.empty():
            logging.warning("No data available, retrying...")
            time.sleep(60)
            continue

        close_vals = self.data['close'].values
        if len(close_vals) < self.window_size:
            logging.warning("Insufficient data for prediction")
            time.sleep(60)
            continue

        # Apply filtering
        if self.use_ralfl.get():
            filtered_data = ralfl_filter_forecast(close_vals)
        else:
            filtered_data = arima_plus_forecast(close_vals, 11, 3)

        # Normalize data for LSTM
        mean, std = np.mean(filtered_data), np.std(filtered_data)
        if std == 0:
            std = 1e-10
        norm_data = (filtered_data - mean) / std
        input_seq = norm_data[-self.window_size:].reshape(1, self.window_size, 1)

        # Predictions
        arima_pred = np.polyfit(range(self.window_size), filtered_data[-self.window_size:], 1)[0] * 1 + filtered_data[-1]
        lstm_pred = self.model.predict(input_seq, verbose=0)[0][0] * std + mean
        combined_pred = (arima_pred + lstm_pred) / 2

        # Get real-time price
        current_price = get_real_time_price(self.client)
        if current_price is None:
            time.sleep(60)
            continue

        # Generate and execute signal
        signal, self.trailing_price = generate_signal(current_price, combined_pred, self.trailing_price)
        if signal != 'hold':
            order = execute_trade(self.client, signal)
            report = f"Signal: {signal}\nPrice: {current_price:.2f}\nPredicted: {combined_pred:.2f}\nOrder: {order}"
            send_email("Trading Bot Report", report)
            send_telegram(report)

        # Update GUI
        self.price_label.config(text=f"{current_price:.2f}")
        self.pred_label.config(text=f"{combined_pred:.2f}")
        self.ax.clear()
        self.ax.plot(self.data.index[-100:], close_vals[-100:], label='Close Price')
        self.ax.plot(self.data.index[-100:], filtered_data[-100:], label='Filtered', linestyle='--')
        self.ax.axhline(y=combined_pred, color='r', linestyle='--', label='Prediction')
        self.ax.legend()
        self.ax.set_title('BTC Price')
        self.canvas.draw()

        time.sleep(60)

# Entry Point
if __name__ == "__main__":
    try:
        client = Client(API_KEY, API_SECRET)
        bq_client = bigquery.Client()
        model = create_lstm_model(WINDOW_SIZE)
        root = tk.Tk()
        root.title("BTC Trading Bot")
        app = TradingBotGUI(root, client, bq_client, model, WINDOW_SIZE)
        root.mainloop()
    except Exception as e:
        logging.error(f"Startup error: {e}")

```